

Explicit energy-conserving modification of relativistic PIC method

Arkady Gonoskov

Department of Physics, University of Gothenburg, SE-41296 Gothenburg, Sweden

ARTICLE INFO

Keywords:

Particle-in-cell (PIC)
Plasma simulation
Energy conservation

ABSTRACT

The use of explicit particle-in-cell (PIC) method for relativistic plasma simulations is restricted by numerical heating and instabilities that may significantly constrain the choice of time and space steps. To partially eliminate these limitations we consider a possibility to enforce exact energy conservation by altering the standard time step splitting. Instead of updating particles in a given field and then the field using the current they produce, we consider subsystems that describe the coupling of a single particle and the field at the nearby nodes and solve them with enforced energy conservation sequentially for all particles, which is completed by the field update with zero current. Such an approach is compatible with various advances, ranging from accounting for additional physical effects to the use of numerical-dispersion-free field solvers, high-order weighting shapes and particle push subcycling. To facilitate further considerations and use, we provide a basic implementation in a 3D, relativistic, spectral code π -PIC, which we make publicly available. The method and its implementations are verified using simulations of cold plasma oscillations, Landau damping and relativistic two-stream instability. The capabilities of the method to deal with large time and space steps are demonstrated in the problem of plasma heating by intense incident radiation.

1. Introduction

The particle-in-cell (PIC) method [1] has a long history of developments that gradually made it suitable for a vast range of numerical studies in fundamental and applied plasma physics. Some of the developments are related to efficient use of computational resources and the possibility to perform computations in parallel on modern supercomputers [2–8]. Other developments concern the inclusion of physical processes, such as ionization or the effects due to strong-field quantum electrodynamics [9–16], as well as the solution of the concomitant algorithmic difficulties, such as particle ensembles resampling [17–30] to compensate for the growth of particle number due to ionization or particle/photon production. Finally there are developments related to the elimination of fundamental limitations, which can potentially facilitate a broad range of PIC method applications.

One such limitation, which remains a long-standing problem, is the numerical dispersion of field solvers. Although dispersion-free spectral solvers have been known from late 70's [31,32], their practical use is limited by difficulties of arranging parallel computations. That is why parallel codes are commonly based on grid-based field solvers (exceptions include codes described in Refs. [8,33,34]). One way to overcome the limitation while still using grid-based methods is to modify the finite-difference time-domain (FDTD) scheme [35] so that the numerical dispersion is suppressed/eliminated along the directions of interest [36–40].

One other long-standing problem of the PIC method is the lack of exact energy conservation, which leads to numerical heating and instabilities [1], thereby restricting the choice of time and space steps. The early developments related to energy-conserving

E-mail address: arkady.gonoskov@physics.gu.se.

<https://doi.org/10.1016/j.jcp.2024.112820>

Received 7 February 2023; Received in revised form 18 January 2024; Accepted 30 January 2024

Available online 5 February 2024

0021-9991/© 2024 The Author(s). Published by Elsevier Inc. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

PIC methods were carried out in early 70's by Lewis [41] and Langdon [42]. The idea of energy-conserving simulation is that the state of particles and the state of electromagnetic field have to be advanced in time concurrently so that the energy conservation is enforced. This can be done by implicit scheme derived from Lagrangian formulation. Nevertheless this leads to the necessity of solving nonlinear system of equations, those number grows with the number of particles. Using Newton Krylov solver makes it possible to exactly preserve energy, under assumption of convergence. This approach was developed and used for the implementation of implicit PIC solvers in Refs. [43–47]. In 2020 Chen, Chacon, and their collaborators proposed an energy- and charge-conserving semi-implicit algorithm that uses multiple implicit iterations for each particle to iteratively solve a system that describes energy coupling between particles and fields (the focus on this coupling is also fundamental to the method we develop here) [48]. Some recent developments on relativistic energy-conserving implicit PIC method are reported in Ref. [49].

In 2017 Lapenta proposed a semi-implicit method (referred to as Energy Conserving Semi-Implicit Method (ECSIM)) that achieves the property of exact energy conservation by solving a linear system with the size proportional to the number of grid nodes, rather than presumably much larger number of particles as in fully implicit methods [50]. The idea of Lapenta method is to include the linear response of particles to the field into Maxwell's equations solved using θ -scheme [51,52] and advance particles using explicit for coordinate and implicit for velocity leap-frog scheme with electric field at the θ time level [53,54]. Although in Ref. [50] the method was developed for non-relativistic case, it is possible that the approach can be generalized to relativistic case and be useful even in highly non-linear simulations.

Nevertheless, for many studies that concern highly relativistic non-linear plasma dynamics, it is common to use explicit PIC method with sufficiently small time and space steps to avoid numerical heating and instabilities (these effects can be also suppressed by high-frequency field filtering). The choice of explicit methods can be related to the readiness for parallel implementation and overall possibility to use smaller time and space steps than that possible with implicit methods under similar computational demands.

In this paper we describe a possibility to modify the standard relativistic PIC scheme so that we enforce exact energy conservation while still keeping the scheme explicit and permitting the use of dispersion-free spectral field solver. This is achieved by splitting the time step into two actions: current-free energy-conserving advancement of the field and energy-conserving local coupling of the field (current-related part) with the dynamics of each particle. The coupling is performed sequentially for all particles, which can be seen as a multiple application of the Strang splitting [55,56]. Although this significantly restricts opportunities for parallel processing, there is still some possibility for parallel implementation, which we describe in Sec. 6 and use in π -PIC code. We discuss options for achieving the desired coupling and describe a basic option of corresponding particle-field pusher. In the limit of small time step the scheme tends to the update of particles' momenta and the current-related change of the field prescribed by the conventional PIC method. However, by performing the particle momentum update together with the update of the electric field at the nearby nodes, we ensure that the total energy remains unchanged to within machine accuracy. We should note that the exact conservation of energy can prevent numerical heating but not necessarily all numerical instabilities, for example finite-grid (or aliasing) instabilities for drifting plasmas (see details in Ref. [57] and in Chapter 8 in the book by Birdsall and Langdon [1]). In summary the proposed method has the following properties:

- the method preserves energy even in case of highly nonlinear, relativistic dynamics and thus is not subject to numerical heating;
- the method doesn't include implicit parts and admits parallel implementation;
- the method is compatible with an arbitrary field solver, including the spectral solver, which is free from numerical dispersion;
- the method admits an implementation that has the second order accuracy in time.

We conjecture that due to these properties the approach described in this paper might be found useful in various numerical studies of relativistic, as well as non-relativistic plasma dynamics.

The paper is arranged as follows. In Sec. 2 we motivate and describe the modified PIC method. In Sec. 3 we derive one basic implementation of the energy-conserving particle-field pusher used in the method and discuss possible improvements. The use of the standard spectral solver is detailed in Sec. 4 for completeness. In Sec. 5 we analyze the accuracy order, develop the variant with the second order accuracy and discuss other possible improvements. In Sec. 6 the basic implementations of the method included in π -PIC are described and tested using the simulation of Landau damping and relativistic two-stream instability. We study basic properties of the method using plasma oscillations in Sec. 7. In Sec. 8 we present a more sophisticated verification test and show the capabilities of the proposed method. We summarize the work in Sec. 9.

2. Modified PIC scheme

When particle dynamics becomes relativistic the idea of enforcing energy conservation faces several difficulties. The first difficulty is that the velocity response becomes nonlinear due to Lorentz factor. One way to combat this is to linearize the response near the current state. Nevertheless, it is possible that the change of particle momentum and velocity over a single time step is not small enough. Recent studies showed [58] that in some cases of interest strong magnetic fields in combination with low particle energies cause difficult-to-resolve gyration, thus requiring advanced particle pushers [59] and subcycling [60]. Although this aspect is clearly important for ensuring accurate numerical results, it doesn't seem the primary difficulty for the energy conservation because even standard Boris pusher [61] preserves energy when accounting only for the magnetic field.

The second difficulty is that over a single time step Δt a relativistic particle can traverse a distance comparable with the space step Δx . (If the spectral field solver is used, the time step can be equal or even larger than $\Delta x/c$, where c is the speed of light.) In case of significant change of particle velocity over Δt , the coordinate becomes another nonlinear part in the response to the field

(along with the velocity itself). Moreover, the update of coordinate by an explicit method based on “old” velocity (as Lapenta method prescribes) unambiguously determines the current, depriving us of the freedom critical for enforcing energy conservation. This is because the current has to be consistent with the charge relocation to comply with the current continuity equation. This consistency is known to be important in relativistic simulations and is perfectly addressed by Esirkepov scheme for current deposition [62]. (For spectral solvers, there are also options of corrections via direct solution of the Poisson’s equation.)

To identify requirements and possibilities for enforcing energy conservation it is instructive to consider 1D problem of electric field interacting with a moving charged particle. Within a sufficiently large time step the energy conservation can become intricate in two different ways: (1) the particle is being accelerated and this process halts when the field energy is exhausted and (2) the field is being set up by the moving particle and this totally exhausts its energy turning the particle in the opposite direction. If both happens within a single step, an oscillating behavior occurs. This consideration indicates one notable point: to achieve energy-conservation it is not possible to advance particle location based on the “old” momentum because we do not know in advance whether the particle will propagate that far. That is why we choose to reverse the common logic: we first determine current that is consistent with momentum change and then determine the next particle location to satisfy the charge continuity equation.

To implement the outlined principle we start from the standard formulation of coupled evolution of classical electromagnetic field and charged particles (CGS units are used):

$$\frac{\partial}{\partial t} \mathbf{B} = -c \nabla \times \mathbf{E}, \quad (1)$$

$$\frac{\partial}{\partial t} \mathbf{E} = c \nabla \times \mathbf{B} - 4\pi \mathbf{J}, \quad (2)$$

$$\mathbf{J} = \sum_{i=1}^N q_i \mathbf{v}_i w(\mathbf{r}_i - \mathbf{r}), \quad (3)$$

$$\frac{\partial}{\partial t} \mathbf{r}_i = \mathbf{v}_i = c \mathbf{p}_i (1 + p_i^2)^{-1/2}, \quad i = 1, \dots, N, \quad (4)$$

$$\frac{\partial}{\partial t} \mathbf{p}_i = \frac{q_i}{m_i c} \left(\mathbf{E}(\mathbf{r}_i) + \frac{1}{c} \mathbf{v}_i \times \mathbf{B}(\mathbf{r}_i) \right), \quad i = 1, \dots, N, \quad (5)$$

where $\mathbf{B}$, $\mathbf{E}$ are magnetic and electric field vectors; $\mathbf{J}$ is the current; $\mathbf{r}_i$, $\mathbf{p}_i$, $\mathbf{v}_i$, q_i and m_i are the i -th particle coordinate, momentum (given in units of $m_i c$), velocity, charge and mass, respectively; N is the number of particles; $w(\cdot)$ quantifies particles’ form factor. To enforce energy conservation we consider time step splitting into steps that advance the state according to subsystems of our choice. We first notice that the field evolution without current do not directly concern the energy coupling between particles and fields. This defines a useful choice for the first subsystem:

$$\frac{\partial}{\partial t} \mathbf{B} = -c \nabla \times \mathbf{E}, \quad (6)$$

$$\frac{\partial}{\partial t} \mathbf{E} = c \nabla \times \mathbf{B}, \quad (7)$$

which can be exactly (to within errors related to the limited number of Fourier modes for the discrete grid) solved using spectral solver to advance the state of field with exact energy conservation (details are given in Sec. 4). Since magnetic field does no work, the effect of magnetic field on momentum $\mathbf{p}_i$ is another part that doesn’t concern energy exchange and defines a useful set of independent subsystems:

$$\frac{\partial}{\partial t} \mathbf{p}_i = \frac{q_i}{m_i c} (1 + p_i^2)^{-1/2} \mathbf{p}_i \times \mathbf{B}(\mathbf{r}_i), \quad i = 1, \dots, N. \quad (8)$$

Each subsystem (8) can also be solved exactly by rotating $\mathbf{p}_i$ about $\mathbf{B}(\mathbf{r}_i)$, or approximately, using Boris scheme, which also preserves energy.

The remaining parts account for the current in Maxwell’s equations and for the electric part of the Lorentz force in the equation of motion. These parts are responsible for the energy coupling between the field and particles. We now notice that we can split the current into contributions of each particle $\mathbf{J}_i$ and separate the remaining subsystem into N subsystems. The i -th subsystem describes local coupling between the field and i -th particle:

$$\frac{\partial}{\partial t} \mathbf{p}_i = \frac{q_i}{m_i c} \mathbf{E}(\mathbf{r}_i), \quad (9)$$

$$\frac{\partial}{\partial t} \mathbf{E} = -4\pi \mathbf{J}_i, \quad (10)$$

$$\mathbf{J}_i = q_i c \mathbf{p}_i (1 + p_i^2)^{-1/2} w(\mathbf{r}_i - \mathbf{r}), \quad (11)$$

$$\frac{\partial}{\partial t} \mathbf{r}_i = c \mathbf{p}_i (1 + p_i^2)^{-1/2} \quad (12)$$

Even though the particles can overlap we intentionally split the coupling between the field and particles into N independent subsystems, which we choose to advance sequentially to be able to enforce energy conservation for each subsystem. This corresponds to the principle of particle-mesh approach that implies that particles do not interact with each other directly within a single time

step, but instead interact through the field defined on a mesh/grid. In what follows we omit index i because the subsystems for each particle are independent and their consideration is identical.

To construct an energy-conserving numerical method for the outlined step splitting we need to formulate a consistent form of subsystem (9) – (12) for the field defined on a grid. Although a complete treatment with advanced approaches can be useful here, we are primarily aiming at preserving energy and thus use the following simplification. We assume that the interaction with the nearest grid values of the field is represented by coefficients $c_j > 0$ that do not change in time during a single time step:

$$\forall \tau \in [t, t + \Delta t] : \mathbf{E}(\mathbf{r}(\tau), \tau) = \sum_{j=1}^M c_j \mathbf{E}_j(\tau), \quad (13)$$

where index j enumerates the nearest M grid nodes involved in the interaction and $\mathbf{E}_j(\tau)$ is the electric field vector at the j -th node. Certainly, the normalization condition must be satisfied:

$$\sum_{j=1}^M c_j = 1. \quad (14)$$

The coefficients are defined by the particle form factor and can be determined in different ways depending on necessary accuracy. In general we need to compute a cumulative overlap. This can be done assuming that the particle propagates with the “old” velocity:

$$c_j = c_j(\mathbf{r}(t), \mathbf{p}(t)) = \Delta t^{-1} \int_{\tau=0}^{\Delta t} d\tau \iiint_{V_g} d^3 r_g S \left(\mathbf{r}(t) + \tau \frac{c\mathbf{p}(t)}{\sqrt{1+p^2(t)}} - \mathbf{r}^{(j)} - \mathbf{r}_g \right), \quad (15)$$

where $S(\mathbf{r})$ describes the shape of the particle, V_g denotes the volume of each cell centered at the j -th node $\mathbf{r}^{(j)}$. To comply with eq. (14) we require that function $S(\mathbf{r})$ is normalized, i.e. $\iiint d^3 r S(\mathbf{r}) = 1$. If the integration can be done analytically the procedure becomes computationally cheap. Alternatively one can approximate the value of the integral using one or several points within the time interval. One can further increase accuracy of the final particle location at the end of the time step by running a predictor-corrector cycle, using the entire procedure described below. Nevertheless, it is not clear whether such more accurate calculations with non-propagating field within one time-step would be more efficient than the overall reduction of the time step for the simplest method. Since our primary goal is to consider energy conservation, in the provided implementation we use the simplest option of integral estimation based on the middle point and cloud-in-cell (CIC) weighting for the shape function.

The method is applicable for arbitrary weighting scheme. To exemplify the use of the method let us detail the case of CIC weighting, which we use in the provided implementation. To do so we define the CIC weighting function:

$$W_{CIC}(\mathbf{r}) = T(x/\Delta x) T(y/\Delta y) T(z/\Delta z), \quad (16)$$

where x, y, z are the Cartesian components of $\mathbf{r}$; $\Delta x, \Delta y, \Delta z$ are the corresponding space steps ($V_g = \Delta x \Delta y \Delta z$), and $T(\cdot)$ is the triangular shape function:

$$T(u) = \begin{cases} 1 - |u|, & |u| < 1, \\ 0, & |u| \geq 1. \end{cases} \quad (17)$$

Assuming that coefficients c_j are fixed for the duration of time step, we define a grid-base version of subsystem (9) – (11):

$$\frac{\partial}{\partial t} \mathbf{p} = \frac{q}{mc} \sum_{j=1}^M c_j \mathbf{E}_j, \quad (18)$$

$$\frac{\partial}{\partial t} \mathbf{E}_j = -4\pi c_j \bar{\mathbf{J}}, \quad j = 1, \dots, M, \quad (19)$$

$$\bar{\mathbf{J}} = \frac{qc}{V_g} \frac{\mathbf{p}}{\sqrt{1+p^2}}, \quad (20)$$

where $\bar{\mathbf{J}}$ has the meaning of the current produced by the particle. To demonstrate that this formulation is consistent in terms of energy conservation, let us explicitly show that the energy is preserved. To do so we multiply both sides of (19) on $\mathbf{E}_j$, sum over all $j = 1, \dots, M$, substitute (20) and (18), and finally obtain

$$\sum_{j=1}^M \mathbf{E}_j \cdot \frac{\partial}{\partial t} \mathbf{E}_j = -4\pi \frac{mc^2}{V_g} \frac{\mathbf{p}}{\sqrt{1+p^2}} \cdot \frac{\partial \mathbf{p}}{\partial t}. \quad (21)$$

Multiplying both sides on $V_g/4\pi$, we finally get

$$\frac{\partial}{\partial t} \left(\frac{V_g}{8\pi} \sum_{j=1}^M E_j^2 + mc^2 \sqrt{1+p^2} \right) = 0, \quad (22)$$

which explicitly shows that the system (18) – (20) preserves total energy of the particle and the field defined on a grid. This means that if we advance the state according (18) – (20) so that the energy is preserved, then the whole method preserves energy.

Now let us show that eqs. (18) – (20) correspond to a well-posed problem, i.e. define a system that has a single solution for arbitrary initial conditions. To show this we substitute (20) into (19), multiply both sides by c_j and sum over all $j = 1, \dots, M$:

$$\sum_{j=1}^M c_j \frac{\partial \mathbf{E}_j}{\partial t} = -4\pi \frac{qc}{V_g} \left(\sum_{j=1}^M c_j^2 \right) \frac{\mathbf{p}}{\sqrt{1+p^2}} = -4\pi \frac{qc}{V_g} \xi \frac{\mathbf{p}}{\sqrt{1+p^2}}, \quad (23)$$

where we defined a constant

$$\xi = \sum_{j=1}^M c_j^2 > 0. \quad (24)$$

We then take time derivative of (18) and substitute (23) into the right hand side:

$$\frac{\partial^2}{\partial t^2} \mathbf{p} = -4\pi \frac{q^2}{mV_g} \xi \frac{\mathbf{p}}{\sqrt{1+p^2}}. \quad (25)$$

Introducing another constant

$$\eta = 4\pi \frac{q^2}{mV_g} \xi > 0, \quad (26)$$

we transform eq. (25) into the form

$$\ddot{\mathbf{p}} = -\nabla_p \left(\eta \sqrt{1+p^2} \right), \quad (27)$$

where we used dot to denote time derivative. In what follows we use the following notations: superscript “+” denotes that the value is taken at the time instance $t + \Delta t$, the absence of such superscript indicates that the value is taken at time instance t , and Δ is used to denote the difference, e.g. $\Delta \mathbf{p} = \mathbf{p}^+ - \mathbf{p} = \mathbf{p}(t + \Delta t) - \mathbf{p}(t)$ (in some cases we indicate the time instance explicitly to avoid confusion).

As we can see (27) is the equation that describes classical motion of an *abstract* particle with coordinate $\mathbf{p}$ in a central potential $\eta\sqrt{1+p^2}$. For the time interval Δt this equation defines a definite solution $\mathbf{p}^+$, $\dot{\mathbf{p}}^+$ for arbitrary initial conditions:

$$\mathbf{p} = \mathbf{p}(t), \quad \dot{\mathbf{p}} = \frac{q}{mc} \sum_{j=1}^M c_j \mathbf{E}_j. \quad (28)$$

We consider the solution of (27) in the next section, while here we complete the proof of consistency by showing that its solution unambiguously and consistently determines $\mathbf{E}_j^+$ and $\mathbf{r}^+$. To show this let us express the change of coordinate over the entire time step $\Delta \mathbf{r} = \mathbf{r}^+ - \mathbf{r}$ using eq. (12):

$$\Delta \mathbf{r} = \int_t^{t+\Delta t} \frac{c\mathbf{p}(\tau)}{\sqrt{1+p^2(\tau)}} d\tau. \quad (29)$$

Substituting (20) into (19) and integrating over the time step we get the change of $\mathbf{E}_j$:

$$\Delta \mathbf{E}_j = \mathbf{E}_j^+ - \mathbf{E}_j = -4\pi c_j \frac{q}{V_g} \int_t^{t+\Delta t} \frac{c\mathbf{p}(\tau)}{\sqrt{1+p^2(\tau)}} d\tau = c_j \Delta \mathbf{E}, \quad (30)$$

where we denote:

$$\Delta \mathbf{E} = -\frac{4\pi q}{V_g} \Delta \mathbf{r}. \quad (31)$$

Considering (18) at time instance $t + \Delta t$ we get:

$$\dot{\mathbf{p}}^+ = \frac{q}{mc} \sum_{j=1}^M c_j (\mathbf{E}_j + c_j \Delta \mathbf{E}). \quad (32)$$

Using (29) – (32) we can express the change of field and the change of coordinate via $\dot{\mathbf{p}}^+$:

$$\Delta \mathbf{E} = \xi^{-1} \left(\frac{mc}{q} \dot{\mathbf{p}}^+ - \sum_{j=1}^M c_j \mathbf{E}_j \right), \quad (33)$$

$$\Delta \mathbf{E}_j = c_j \Delta \mathbf{E}, \quad (34)$$

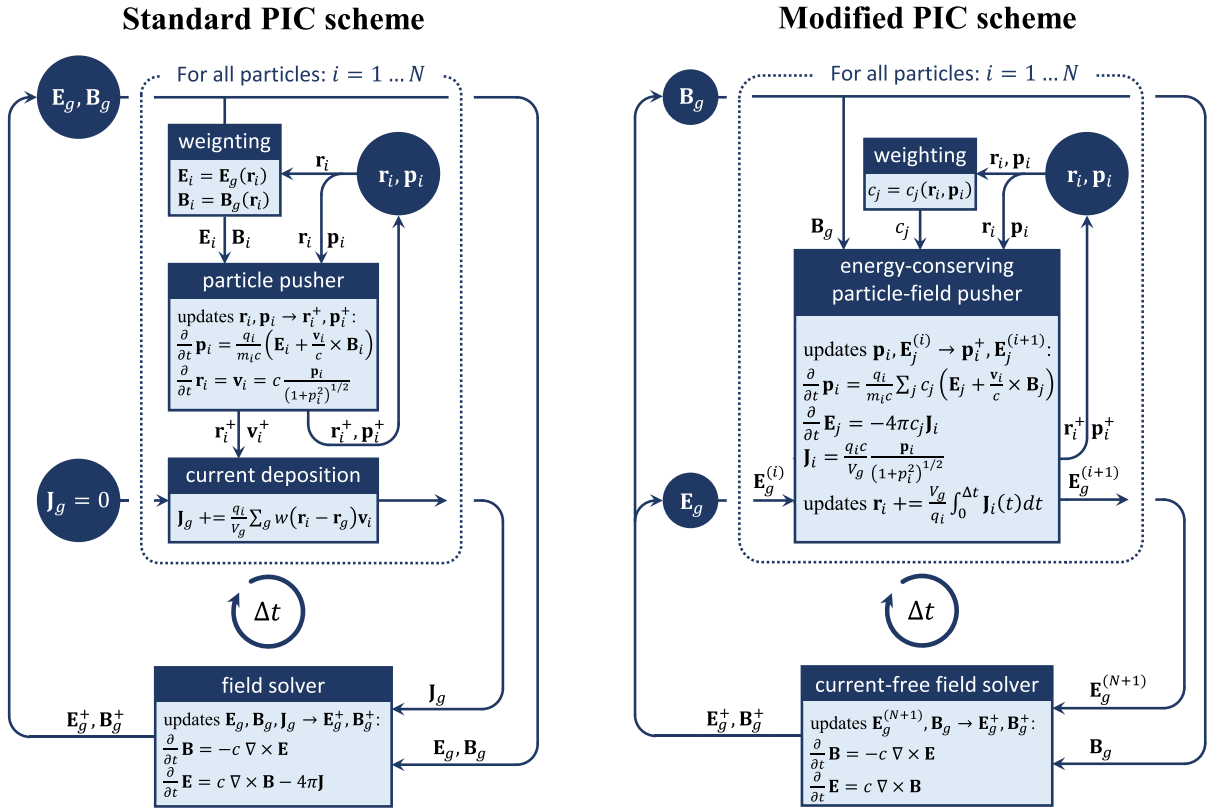


Fig. 1. The standard (left) and modified (right) PIC schemes. For the modified scheme $\mathbf{E}^{(i)}$ denotes the electric field updated by the particle-field pusher applied for particles with indices up to $i - 1$.

$$\Delta \mathbf{r} = -\frac{V_g}{4\pi q} \Delta \mathbf{E}. \quad (35)$$

Thus our task is to find $\mathbf{p}^+$ and $\mathbf{p}^+$ according to eq. (27), preserving energy given by (22). The solution of this task in combination with updates (33) – (35) and the momentum update according to (8) can be referred to as *energy-conserving particle-field pusher*. We consider one possible implementation of such pusher in the next section.

We conclude this section by summarizing the proposed modification of the PIC method. The method implies that we sequentially advance the state of each particle and of electric field at the coupled nearby nodes according to (18) – (20). Apart from this for each particle we advance the momentum according to the local magnetic field using rotation about the magnetic field vector. Finally, after advancing the states of all particles together with current-related change of the electric field at the coupled nodes, we use spectral solver to advance the field state according to Maxwell's equations without current. In Fig. 1 we show the standard PIC scheme and the described modification in a way that highlights the differences: the circles are used to denote data, while the data processing principles are summarized in rectangular elements; the superscript “+” indicates the updated data; subscript “g” denotes grid values, $\mathbf{E}_g^{(i)}$ and $\mathbf{E}_g^{(i+1)}$ denote the state of electric field before and after processing the i -th particle, respectively.

3. Explicit energy-conserving particle-field pusher

The energy-conserving particle-field pushed introduced in Fig. 1 can be generally defined as an algorithm that advances the state of a given particle together with the state of the field at the nearby nodes according to both (8) and (9) – (12). An exact advancement warrants energy conservation given by eq. (22). For approximate solution we need to enforce energy conservation. Note that improved properties that reduce violation of energy conservation can be also practical because they can broaden the parameter region of numerical stability. However we here develop a solution with exact energy conservation.

Although a more comprehensive treatment can be useful, we here use several simplifications to get an approximate, yet energy-conserving solution of this problem. First, as mentioned above, we separate and split the effect of magnetic field, which we account for through the rotation of $\mathbf{p}$ about $\mathbf{B}$ before the advancement according to eqs. (9) – (12); the details are given in the summary of the algorithm below. Next, we assume that coefficients c_j are not changing in time during each time step. Under these assumptions, given relations (33) – (35), the remaining task is to advance the state of $\mathbf{p}(t)$ and $\mathbf{p}(t)$ defined by (28) over one time step according to (27):

$$\dot{\mathbf{p}} = -\nabla_p \left(\eta \sqrt{1+p^2} \right), \quad \eta > 0. \quad (36)$$

This equation can be seen as an equation that describes motion of an *abstract* non-relativistic particle with coordinate p in a central potential $\eta \sqrt{1+p^2}$. Hereafter we refer to this particle as *p-particle*, so as not to confuse it with the real particle in question.

The relation to the p-particle dynamics provides an insight into the origin of numerical heating and corresponding instability. In case of exact dynamics, p-particle is constantly being deflected towards the center, while in the standard PIC scheme the p-particle is advanced in time along a straight line pointing towards $\dot{\mathbf{p}}(t)$ (tangent to the trajectory at the initial point of the time step). In case of a small enough time step, the deviation is minor but always leads to p-particle being further from the center than it should be, meaning that the energy is constantly increased. We thus can conclude that accounting for the potential that restricts p-particle motion is the way to get rid of numerical heating.

For the motion of p-particle we have two invariants:

$$\mathbf{p} \times \dot{\mathbf{p}} = L = \text{const}, \quad (37)$$

$$\frac{1}{2} |\dot{\mathbf{p}}|^2 + \eta \sqrt{1+p^2} = \mathcal{E} = \text{const}, \quad (38)$$

where the first one represents the angular momentum conservation and the second one represent the energy conservation. Note that, despite having similar terms proportional to $\sqrt{1+p^2}$, the energy conservation for p-particle differs from the energy conservation for the field and the real particle given by eq. (22).

In case of eq. (36), the standard way of determining particle dynamics in a central potential by adding centrifugal potential leads to differential equation [63]:

$$\dot{p} = \sqrt{2\mathcal{E} - L/p^2 - 2\eta \sqrt{1+p^2}}. \quad (39)$$

Since the exact solution is intricate, we choose to use another simplification with the following plan. We use another potential that approximates the one in eq. (36), but leads to an explicit solution. We then have two main options to enforce exact energy conservation according to eq. (22). We can either use approximate $\mathbf{p}^+$ and correct the approximate $\Delta \mathbf{E}(\dot{\mathbf{p}}^+)$ (see eq. (33)), or use approximate $\dot{\mathbf{p}}^+$, determine $\Delta \mathbf{E}(\dot{\mathbf{p}}^+)$ and then correct the approximate $\mathbf{p}^+$. Since the correction concerns 3D vector based on a single equation, there are many options. We choose to correct $\mathbf{p}^+$ keeping its direction unchanged, i.e. by multiplying vector $\mathbf{p}^+$ by a factor σ , which presumably should be close to unit. Since the correction of particle energy concerns the length of $\mathbf{p}$ only, the vector of difference between the approximate $\mathbf{p}^+$ and the corrected $\sigma \mathbf{p}^+$ has the minimal length among all possible ways of correcting $\mathbf{p}^+$ (determining such a minimal correction for $\Delta \mathbf{E}$ is also possible but requires some derivations). The condition for enforced energy conservation then takes the form:

$$\frac{V_g}{8\pi} \sum_{j=1}^M E_j^2 + mc^2 \sqrt{1+p^2} = \frac{V_g}{8\pi} \sum_{j=1}^M E_j^{+2} + mc^2 \sqrt{1+\sigma^2 p^{+2}}. \quad (40)$$

We can explicitly express the correction factor σ :

$$\sigma = \left(\left(\left(\frac{V_g}{8\pi mc^2} \sum_{j=1}^M (E_j^2 - E_j^{+2}) + \sqrt{1+p^2} \right)^2 - 1 \right) p^{+2} \right)^{1/2}. \quad (41)$$

There is, however, an important restriction on the choice of approximate potential. It should restrict the motion of p-particle in such a way that the correction is always possible. In our case, it means that the resultant approximate $\dot{\mathbf{p}}^+$ doesn't lead to the field taking more energy than the particle can provide, otherwise we get imaginary value for σ .

Lemma. *If $\dot{p}^+$ is limited by the maximal kinetic energy gain for p-particle then σ is real.*

Proof. The restriction on the maximal kinetic energy gain for p-particle can be expressed as

$$(\dot{p}^+)^2 / 2 \leq \eta (\sqrt{1+p^2} - 1) + p^2 / 2, \quad (42)$$

which is achieved for $L = 0$, $p^+ = 0$. Let us now determine the upper bound for the electric field energy gain:

$$\Delta \mathcal{E}_E = \frac{V_g}{8\pi} \sum_{j=1}^M (E_j^{+2} - E_j^2) = \frac{V_g}{8\pi} \sum_{j=1}^M (2c_j \mathbf{E}_j \Delta \mathbf{E} + c_j^2 \Delta E^2), \quad (43)$$

where we substituted $\mathbf{E}_j^+ = \mathbf{E}_j + c_j \Delta \mathbf{E}$ given eqs. (33) – (34). Since $c_j > 0$, the largest value is achieved in case all $\mathbf{E}_j$ are pointing along $\Delta \mathbf{E}$, therefore

$$\Delta \mathcal{E}_E \leq \frac{V_g}{8\pi} \sum_{j=1}^M (2c_j E_j \Delta E + c_j^2 \Delta E^2) = \frac{V_g}{8\pi} \left(2\Delta E \sum_{j=1}^M c_j E_j + \xi \Delta E^2 \right). \quad (44)$$

Substituting eq. (33), we get

$$\Delta \mathcal{E}_E \leq \frac{V_g m^2 c^2}{8\pi \xi q^2} (\dot{p}^+)^2 - \frac{V_g}{8\pi \xi} \left(\sum_{j=1}^M c_j E_j \right)^2. \quad (45)$$

Substituting restriction (43) and expressing $\dot{p}^2$ via eq. (18), we obtain

$$\Delta \mathcal{E}_E \leq mc^2 \left(\sqrt{1+p^2} - 1 \right). \quad (46)$$

Recalling the definition of $\Delta \mathcal{E}_E$ (43) and substituting inequality (46) into eq. (41), we can see that radicand is greater or equal to zero and thus σ is real. $\square$

Given the obtained restriction we choose to approximate the exact potential $\eta \sqrt{1+p^2}$ by a parabolic potential $\kappa p^2/2$, where coefficient κ is chosen to match the slope at the initial point $\mathbf{p}(t)$:

$$\kappa = \frac{\eta}{\sqrt{1+p(t)^2}}. \quad (47)$$

This potential satisfies the identified requirement because the maximal drop of potential energy (reached at $p=0$) is less than that of the exact potential. This follows from the fact that the gradient on the way to $p=0$ is always smaller for the approximate potential:

$$\forall p \leq p(t) : \left| \nabla_p \left(\eta \sqrt{1+p^2} \right) \right| \leq \left| \nabla_p \left(\kappa p^2/2 \right) \right|. \quad (48)$$

Note that the potentials exactly match in the limit $p^2 \ll 1$.

The equation for the dynamics of p-particle with the approximate potential takes the form of three dimensional symmetric harmonic oscillator:

$$\ddot{\mathbf{p}} = -\kappa \mathbf{p}. \quad (49)$$

To express the solution in explicit form we define a function

$$\mathbf{g} = \dot{\mathbf{p}} + i\sqrt{\kappa} \mathbf{p} \quad (50)$$

and compute its time derivative using (49):

$$\dot{\mathbf{g}} = -\kappa \mathbf{p} + i\sqrt{\kappa} \dot{\mathbf{p}} = i\sqrt{\kappa} \mathbf{g}. \quad (51)$$

Using the solution $\mathbf{g}^+ = \mathbf{g} \exp(i\sqrt{\kappa} \Delta t)$, we get

$$\dot{\mathbf{p}}^+ = \text{Re} \left\{ \left(\dot{\mathbf{p}} + i\sqrt{\kappa} \mathbf{p} \right) \exp(i\sqrt{\kappa} \Delta t) \right\}, \quad (52)$$

$$\mathbf{p}^+ = \frac{1}{\sqrt{\kappa}} \text{Im} \left\{ \left(\dot{\mathbf{p}} + i\sqrt{\kappa} \mathbf{p} \right) \exp(i\sqrt{\kappa} \Delta t) \right\}. \quad (53)$$

Note that in case of non-relativistic description the approximate parabolic potential coincides with the exact one. In this case we do not need to do any correction and have exact energy conservation (22) already for (52) – (53) with E_j^+ determined according to (33) – (34). In case of relativistic simulations, the solution (52) – (53) for approximate potential becomes increasingly close to that for exact $\eta \sqrt{1+p^2}$.

To complete the description of this basic form of energy-conserving particle-field pusher we address the question of changing the momentum according to the local magnetic field $\mathbf{B}$:

$$\dot{\mathbf{p}} = \frac{q}{mc} \frac{\mathbf{p}}{\sqrt{1+p^2}} \times \mathbf{B}. \quad (54)$$

For constant magnetic field this equation describes the rotation of vector $\mathbf{p}$ about vector $\mathbf{B}$. Under this assumption we can express the solution using a rotation operator $\hat{R}_{\mathbf{a}} \mathbf{b}$, which rotates vector $\mathbf{b}$ about $\mathbf{a}$ by an angle a . To obtain explicit form of this operator we decompose $\mathbf{b}$ into the sum of parallel and perpendicular components with respect to vector $\mathbf{a}$:

$$\mathbf{b} = \mathbf{b}_{||} + \mathbf{b}_{\perp}, \quad (55)$$

$$\mathbf{b}_{||} = \mathbf{a} \frac{\mathbf{b} \cdot \mathbf{a}}{a^2}, \quad \mathbf{b}_{\perp} = \mathbf{b} - \mathbf{a} \frac{\mathbf{b} \cdot \mathbf{a}}{a^2}. \quad (56)$$

Using this decomposition, we can express the rotation operator:

$$\hat{R}_{\mathbf{a}} \mathbf{b} = \mathbf{b}_{||} + \mathbf{b}_{\perp} \cos a + \frac{\mathbf{a} \times \mathbf{b}}{a} \sin a$$

$$= \mathbf{b} \cos a + \mathbf{a} \frac{\mathbf{b} \cdot \mathbf{a}}{a^2} (1 - \cos a) + \frac{\mathbf{a} \times \mathbf{b}}{a} \sin a. \quad (57)$$

The rotation of $\mathbf{p}$ described by eq. (54) can be given by

$$\mathbf{p}^+ = \hat{R}_{\zeta(p)\mathbf{B}} \mathbf{p}, \quad (58)$$

$$\zeta(p) = \frac{q\Delta t}{mc\sqrt{1+p^2}}. \quad (59)$$

The rotation of momentum $\mathbf{p}$ due to magnetic field doesn't change the energy of the particle. We can apply it before or after coupling with the electric field. One can also split the step into two half-step rotations applied before and after coupling with the electric field. Nevertheless, we leave possible improvements for further considerations elsewhere and use the simplest version of update based on the Boris pusher [64]:

$$\mathbf{t} = \frac{q\Delta t}{2mc\sqrt{1+p^2}} \mathbf{B}, \quad (60)$$

$$\mathbf{u} = \mathbf{p} + \mathbf{p} \times \mathbf{t}, \quad (61)$$

$$\mathbf{p}^+ = \mathbf{p} + \frac{2}{1+t^2} \mathbf{u} \times \mathbf{t}, \quad (62)$$

and denote this algorithm of update by $\mathbf{p}^+ = \hat{R}_{\Delta t \mathbf{B}}^{Boris} \mathbf{p}$.

We can now collect all the steps of such basic energy-conserving particle-field pusher, which for i -th particle converts $\mathbf{r} = \mathbf{r}_i$, $\mathbf{p} = \mathbf{p}_i$ and $\mathbf{E} = \mathbf{E}^{(i)}$ into $\mathbf{r}^+ = \mathbf{r}_i^+$, $\mathbf{p}_i^+ = \mathbf{p}^+$ and $\mathbf{E}^{(i+1)} = \mathbf{E}^+$ (see Fig. 1):

1. $c_j := W_{CIC} \left(\mathbf{r} + \frac{\Delta t}{2} \frac{c\mathbf{p}}{\sqrt{1+p^2}} - \mathbf{r}^{(j)} \right)$, $j = 1, \dots, M$
2. $\mathbf{B} := \sum_j^M c_j \mathbf{B}_j$, $\tilde{\mathbf{p}} := \hat{R}_{\Delta t \mathbf{B}}^{Boris} \mathbf{p}$
3. $\xi := \sum_{j=1}^M c_j^2$, $\kappa := \frac{4\pi q^2}{mV_g \sqrt{1+p^2}} \xi$
4. $\dot{\mathbf{p}} := \frac{q}{mc} \sum_{j=1}^M c_j \mathbf{E}_j$
 $\dot{\mathbf{p}} := \text{Re} \left\{ \left(\dot{\mathbf{p}} + i\sqrt{\kappa} \dot{\mathbf{p}} \right) \exp \left(i\sqrt{\kappa} \Delta t \right) \right\}$
 $\tilde{\mathbf{p}} := \frac{1}{\sqrt{\kappa}} \text{Im} \left\{ \left(\dot{\mathbf{p}} + i\sqrt{\kappa} \dot{\mathbf{p}} \right) \exp \left(i\sqrt{\kappa} \Delta t \right) \right\}$
5. $\Delta \mathbf{E} := \xi^{-1} \left(\frac{mc}{q} \dot{\mathbf{p}} - \sum_{j=1}^M c_j \mathbf{E}_j \right)$
 $\mathbf{E}_j^+ := \mathbf{E}_j + c_j \Delta \mathbf{E}$, $j = 1, \dots, M$
6. $\sigma := \left(\left(\frac{V_g}{8\pi mc^2} \sum_{j=1}^M \left(E_j^2 - E_j^{+2} \right) + \sqrt{1+p^2} \right)^2 - 1 \right)^{1/2} \tilde{\mathbf{p}}^{-2}$
 $\mathbf{p}^+ := \sigma \tilde{\mathbf{p}}$
7. $\mathbf{r}^+ := \mathbf{r} - \frac{V_g}{4\pi q} \Delta \mathbf{E}$

We conclude this section by outlining some limitations and possible improvements of the described algorithm. The presented consideration indicates two characteristic time scales of cyclic behavior. The first one is induced by magnetic field and describes particle gyration with frequency $\omega_B = qB/mc\gamma$ (see eq. (54)), where $\gamma = \sqrt{1+p^2}$ is the particle gamma factor. The frequency of the second one $\sqrt{\kappa}$ (see eq. (49)) corresponds to plasma frequency for the *partial* density. To see this and clarify the meaning of the partial density, we recall that in PIC method the time evolution is computed for macroparticles that are assumed to represent a number of real particles, the number of which is commonly referred to as *weight*. Denoting the real particle mass and charge for the i -th macroparticle with weight w_i by $q_r = q_i/w_i$ and $m_r = m_i/w_i$ respectively, we can represent the frequency $\sqrt{\kappa}$ in the following form (see eqs. (47) and (26)):

$$\sqrt{\kappa} = \sqrt{\frac{4\pi (w_i q_r)^2}{w_i m_r \gamma V_g}} \xi = \sqrt{\frac{4\pi q_r^2 n_i^p}{m_r \gamma}}, \quad (63)$$

where partial density $n_i^p = w_i/(V_g/\xi)$ corresponds to the density that real particles associated the i -th macroparticle would have in case of even distribution in an effective volume V_g/ξ (note that $\xi \sim 1$ for the CIC weighting). The right-hand side of eq. (63) indicates that $\sqrt{\kappa}$ represents the time scale of plasma oscillations with the density of n_i^p and with account for the relativistic mass increase.

The algorithm described in this section provides the state advancement that can be sufficient for simulations even if the outlined time scales of cyclic behavior locally become comparable to or even shorter than the time step. For example, the algorithm can be sufficient if the accurate tracking of the phases of oscillations is not important and the energy conservation is a sufficient property that provides an appropriate modeling of particle's response for the simulation of macroscopic processes in question. Nevertheless, if a more accurate treatment is needed, there are few interesting possibilities for improvement. To reveal such possibilities let us consider in what ways the algorithm can be inaccurate.

First, a poor resolution of particle gyration in magnetic field may lead to inaccurate description of the particle drift and energy gain near stopping points during direct laser acceleration (see [58,60]). To mitigate these effects in relativistic simulations one can apply the approach of *individual subcycling* proposed by Arefiev et al. [60] and further developed in [58]. In the context of described algorithm the individual subcycling means that for each particle we determine an individual time sub-step Δt_i so that $\omega_B \Delta t_i \ll 1$, and repeat steps 1-3 with this time sub-step (if $\Delta t_i \geq \Delta t$ we do a single time step with Δt).

The individual subcycling can also provide a way to account for the changing gamma factor during plasma oscillation. This is related to another source of inaccuracy that is caused by the use of approximate parabolic potential instead of $\eta\sqrt{1+p^2}$. To mitigate this inaccuracy the choice of individual sub-step should be driven by requiring $\Delta t_i \sqrt{\kappa} \ll 1$.

The proposed use of parabolic approximation for the potential in eq. (27) becomes increasingly accurate for low energies (i.e. for $p^2 \ll 1$). That is why the approximation becomes negligible for plasma states with non-relativistic temperatures, while still being reasonable for relativistic dynamics, particularly in the sense of restricting the gain of kinetic energies for particles in case of large time steps. Let us note that the Euler-type momentum advancement in a given electric field (used in the Boris method) can be seen as the replacement of $\eta\sqrt{1+p^2}$ potential with zero potential, in which the p -particle propagates linearly according to the initial $\mathbf{p}$ determined by eq. (28). In this context the proposed parabolic approximation appears to be a good tradeoff between accuracy and computational complexity: instead of developing a scheme with higher order accuracy or even exact treatment of (27) it might be more beneficial to use larger number of particles and/or perform collective subcycling.

One other source of inaccuracy is related to the fact that the simulated plasma frequency (63) corresponds to partial density of a single macroparticle, while a large number of macroparticles for the given space location would imply a higher plasma frequency. This kind of inaccuracy is related to the fact that particles interact via field, not directly. To mitigate this effect, one can apply the following procedure. For each particle we determine a time sub-step that ensures a good resolution of the actual plasma oscillations

$$\Delta t_i \ll \left(\frac{4\pi q_r^2 n_r}{m_r \gamma} \right)^{-1/2}, \quad (64)$$

where n_r is the estimated local density of particles. If $\Delta t_i < \Delta t$ the particle is advanced in time by Δt_i and flagged as a particle with uncompleted time step with individual time $t_i = t + \Delta t_i$. Once all particles are processed, we return back to the flagged particles and advance each again by another individual time sub-step, increasing t_i by Δt_i and unflagging it if $t_i = t + \Delta t$ (the substep Δt_i can either be adjusted to fit a whole number of sub steps in Δt or the last step can be cut to fit the time at the end). We repeat this cycle over flagged particles until they all become unflagged. To highlight the difference in the way of processing particles one can call this *collective subcycling*. It is important that the procedure becomes computationally demanding only locally, where the density is high. Nevertheless, accounting for multiple plasma oscillations within a single time step may still be inaccurate because we do not account for the electromagnetic field evolution during a single time step. Effectively, we inaccurately describe the oscillating current that would otherwise result in energy emission in the form of electromagnetic waves. In a more general case, the plasma oscillations can be strongly coupled with electromagnetic waves. That is why there should be some practical limit for the use of subcycling.

One other reason for resolving the time scale of $\sqrt{\kappa}$ is related to the motion of particles. To see the mechanism of this limitation, let us consider the case, when the time step is so large that we have $\sqrt{\kappa}\Delta t = \pi$ and also let us assume that $\mathbf{p} = 0$. In this case, $\Delta \mathbf{E} = 2\mathbf{E}$ no matter how strong the field is (one can interpret this as a half oscillation for plasma associated with a single particle). Following step 7 of the algorithm this means that the change of coordinate $\mathbf{r}^+ - \mathbf{r} = V_g E / 2\pi q$ can potentially exceed $c\Delta t$. One can interpret this limitation as the necessity to resolve plasma period for partial density created by individual particles. Assuming equal weights of particles, this period is larger than the actual plasma period by a factor that is equal to the square root of the number of particles per cell.

There is a source of inaccuracy that is related to the discrepancy between the c_i determined based on the "old" velocity and the c_i that correspond to the actual change of particle coordinate $\Delta \mathbf{r}$. This is related to the violation of the continuity equation for the current and thus can lead to the accumulation of spurious charge (note that the correction based on solving Poisson's equation is possible but can violate the energy conservation). This kind of inaccuracy becomes increasingly large with more localized shape functions and when the motion is relativistic (and also when the particle is near the stopping points). Although subcycling can also alleviate this difficulty, there is another possibility. In case the determined $\Delta \mathbf{r}$ corresponds to largely different c_i , we can repeat the computations with c_i determined at $\mathbf{r} + \Delta \mathbf{r}/2$. One can even run a cycle of such correcting procedure, expecting that it quickly converges to a consistent advancement.

Finally, there is one other collective source of inaccuracy related to the fact that during each step the particles interact with the field sequentially. This means that field experienced by each particle depends on the order of processing. The effect is small if the time step is small, as is the total field change over each time step. Nevertheless, it is interesting to consider the possibility of

using large time steps. In this case, one can again use local collective subcycling. However, there is also another simple measure to suppress the effect of order. Since the order of processing can be correlated with the particle location in phase space, ordered processing means that different parts of phase space are repeatedly experiencing the fields at the same stage of update after the update by the field solver. To remove the accumulation of such bias we can shuffle the order of processing. This is implemented in the provided implementation, while the positive effect of shuffling is shown in Sec. 7.2.

4. Field advancement using spectral method

To complete the time step so that the energy is exactly preserved we need to do energy-conserving advancement of electromagnetic field. This can be done by standard spectral method that in addition is free of numerical dispersion. Although the spectral method can be considered a well-recognized alternative to finite-difference and finite-element methods (i.e. grid-based methods) it remains a rather uncommon choice in many numerical studies. We thus provide a brief description of the method for completeness.

The spectral solver of Maxwell's equations was described by Haber et al. in 1973 [31], by Buneman et al. in 1980 [32], and later by Birdsall and Langdon in their textbook "Plasma Physics via Computer Simulation" (see chapter 15.9 (b)) published in 1984 [1]. The solver implies that Fast Fourier Transform (FFT) quickly converts the field defined on a grid into a sum of plane wave modes, each of those is independently advanced in time without errors, and then returns the field to the initial coordinate representation on the grid. This gives exact solution (without numerical dispersion) in case of constant current. In 1997 Liu considered an approximate/truncated version called pseudo-spectral time-dependent (PSTD) [65], which doesn't require sine/cosine function computation but is subject to numerical dispersion. In 2013 Vay et al. considered the exact solution (also for time staggered field components) and introduced the term pseudo-spectral analytical time-domain (PSATD), which highlights the difference from PSTD [33]. In the same work Vay et al. proposed a solution to the problem of arranging parallel computations being the main stumbling block for the broad use of the method. Exploiting linearity of Maxwell's equation and the finiteness of the signal propagation speed, the method in parallel advances the field in spatial domains having overlaps, where the cross-summation of the fields is carried out once in a time interval less than that needed for the light to traverse the overlap. A different method for arranging parallel computations, as well as the PIC code ELMIS based on this method, was described and used for numerical studies in PhD thesis [34] also published in 2013. The method is based on alternating data decomposition throughout each computational step (for details see Sec. 6). Although the term PSATD is being used in recent works [8,66], we here revert to the original term *spectral solver* used in the pioneering works [1,31,32].

The spectral solver for Maxwell's equations can be seen as a particular case of a general approach to numerical solution of partial differential equations (PDE). The idea is to use time step splitting so that the time advancement due to linear part (with optional inclusion of constant terms) of a PDE is carried out using eigenbasis so that the time-advancing operational exponent is computed exactly via the Jordan canonical form. In physics the relevant bases for getting useful diagonal representation of the operational exponent correspond to wave and coordinate representations of the state function, which means that the method can be greatly facilitated by the use of FFT. Although general procedure is applicable to the case of Maxwell's equations, it is instructive to consider the derivation of the spectral methods in physical terms. Following Ref. [32], we consider complex-value field that combines electric (real part) and magnetic (imaginary part) field components:

$$\mathbf{F} = \mathbf{E} + i\mathbf{B}, \quad (65)$$

where i is the imaginary unit. For vector $\mathbf{F}$ Maxwell's equations with zero current take the following form:

$$\nabla \times \mathbf{F} = \frac{i}{c} \frac{\partial}{\partial t} \mathbf{F}, \quad (66)$$

$$\nabla \cdot \mathbf{F} = 4\pi\rho, \quad (67)$$

where ρ is the charge density and the current $\mathbf{J}(\mathbf{r})$ is omitted in accordance with the PIC modification in question. Applying Fourier transform to both sides of equations (66), (67) and denoting the representations of functions $\mathbf{F}(\mathbf{r})$ and $\rho(\mathbf{r})$ in the basis of wave-vectors $\mathbf{k}$ by subscript $\mathbf{k}$, we get:

$$\frac{1}{c} \frac{\partial}{\partial t} \mathbf{F}_{\mathbf{k}} = \mathbf{k} \times \mathbf{F}_{\mathbf{k}}, \quad (68)$$

$$\mathbf{k} \cdot \mathbf{F}_{\mathbf{k}} = -4\pi i \rho_{\mathbf{k}}. \quad (69)$$

Eq. (69) corresponds to Poisson's equation and remains true if the current and charge fulfill the continuity equation. The propagation of electromagnetic radiation is described by eq. (68), which describes the rotation of vector $\mathbf{F}_{\mathbf{k}}$ about vector $\mathbf{k}$ with frequency ck . Note that one can also exactly solve the equation for $\mathbf{F}$ with constant current (see [1]), but this is not of interest for us because the current is already accounted for in the particle-field pusher. We thus can express the time advancement via rotation operator given by eq. (57):

$$\mathbf{F}^+ = \hat{R}_{c\Delta t/\mathbf{k}} \mathbf{F}. \quad (70)$$

5. Accuracy order and possible improvements

In this section we demonstrate that the described implementation has only the first order accuracy in time, elaborate a variant of the method that has the second order accuracy and consider other possibilities for improvements. To assess the accuracy order we consider separately three distinctly different aspects.

The first one concerns coupling between the electric field and the momentum of each particle. In the conventional Boris pusher this coupling is not included in the momentum update in the sense that the field is assumed to remain unchanged during the updates (the coupling is enabled via current). This becomes an increasingly accurate assumption with decrease of time step and particle weight (increase of the number of particles per cell). The time staggered update of momentum and field results in the second order accuracy. However, the proposed energy-conserving method aims at handling large time steps and thus we here need to consider a more complicated problem that includes the coupling with the field.

The second aspect concerns the use of parabolic approximation of the potential in eq. (27), which can be also interpreted as the use of an effective relativistic mass increase. For the particle-field coupling the first order accuracy for the update of momentum $\mathbf{p}$ corresponds to solving equation $\dot{\mathbf{p}} = 0$, which takes into account only instantaneous electric field state at the time instance of update. Matching the gradient of the potential in eq. (27) with the parabolic potential leads to the first order accuracy for the update of $\dot{\mathbf{p}}$ and the second order accuracy for the update of $\mathbf{p}$.

The third aspect concerns splitting of the ensemble update into a sequence of updates for all the particles separately. This can be seen as a multiple application of the Strang splitting [55,56] to the coupled system of field and all the particles. To assess the accuracy order and the possibility for its increase let us consider the case of just two particles interacting with the field. Since we already considered the aspect of approximating relativistic potential with an effective non-relativistic potential, we assume here that both particles are non-relativistic. For simplicity we also assume that the particles have the same charges and masses, and are coupled with a single field node. Eqs. (9) – (11) for such a simplified case take the form:

$$\frac{\partial \mathbf{p}_1}{\partial t} = \alpha \mathbf{E}, \quad (71)$$

$$\frac{\partial \mathbf{p}_2}{\partial t} = \alpha \mathbf{E}, \quad (72)$$

$$\frac{\partial \mathbf{E}}{\partial t} = \beta (\mathbf{p}_1 + \mathbf{p}_2), \quad (73)$$

where $\alpha = q/mc$, $\beta = -4\pi qc$. This system can be written in the matrix form:

$$\frac{\partial}{\partial t} \psi = S \psi, \quad (74)$$

where the state vector ψ and matrix S are defined by:

$$\psi = \begin{bmatrix} \mathbf{p}_1 \\ \mathbf{p}_2 \\ \mathbf{E} \end{bmatrix}, \quad S = \begin{bmatrix} 0 & 0 & \alpha \\ 0 & 0 & \alpha \\ \beta & \beta & 0 \end{bmatrix}. \quad (75)$$

The exact solution to this problem for updating the state from $\psi(t)$ to $\psi(t + \tau)$ has the form:

$$\psi(t + \tau) = e^{\tau S} \psi(t). \quad (76)$$

The idea of Strang splitting is based on representing matrix S as a sum of two matrices S_1 and S_2 , each of those describe a subsystem with reduced complexity. In our case the matrices take the form:

$$S = S_1 + S_2, \quad S_1 = \begin{bmatrix} 0 & 0 & \alpha \\ 0 & 0 & 0 \\ \beta & 0 & 0 \end{bmatrix}, \quad S_2 = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & \alpha \\ 0 & \beta & 0 \end{bmatrix}. \quad (77)$$

As one can see the subsystem described by S_1 concerns the update of $\mathbf{p}_1$ and $\mathbf{E}$, whereas the subsystem described by S_2 concerns the update of $\mathbf{p}_2$ and $\mathbf{E}$. Solving these systems sequentially for a small time step τ can be seen as the main idea behind the proposed method labeled below with subscript “ec”:

$$\psi(t + \tau) \approx \psi_{ec}(t + \tau) = e^{\tau S_1} e^{\tau S_2} \psi(t). \quad (78)$$

To determine the accuracy order related to splitting we consider the difference between the exact solution $\psi(t + \tau)$ and the approximate solution given by $\psi_{ec}(t + \tau)$:

$$\psi(t + \tau) - \psi_{ec}(t + \tau) = \left(e^{\tau(S_1+S_2)} - e^{\tau S_1} e^{\tau S_2} \right) \psi(t) = \frac{\tau^2}{2} (S_2 S_1 - S_1 S_2) + O(\tau^3). \quad (79)$$

Since matrices S_1 and S_2 do not commute, the solver based on the introduced splitting has the first order accuracy.

The presented analysis indicates a possibility for increasing the accuracy order. Let us first split the exact solution (76) into two equal sub-steps and then apply splitting for both sub-steps, but using the reversed order in eq. (78) for the second sub-step:

$$e^{\tau S} = e^{\frac{\tau}{2} S} e^{\frac{\tau}{2} S} = e^{\frac{\tau}{2} S_1 + \frac{\tau}{2} S_2} e^{\frac{\tau}{2} S_2 + \frac{\tau}{2} S_1} \quad (80)$$

$$= \left(e^{\frac{\tau}{2} S_1} e^{\frac{\tau}{2} S_2} + \frac{\tau^2}{8} (S_2 S_1 - S_1 S_2) + O(\tau^3) \right) \left(e^{\frac{\tau}{2} S_2} e^{\frac{\tau}{2} S_1} + \frac{\tau^2}{8} (S_1 S_2 - S_2 S_1) + O(\tau^3) \right) \quad (81)$$

$$= e^{\frac{\tau}{2} S_1} e^{\frac{\tau}{2} S_2} e^{\frac{\tau}{2} S_2} e^{\frac{\tau}{2} S_1} + O(\tau^3). \quad (82)$$

Thus, the following expression describes the solver with the second order accuracy, which is labeled with “ec2” below:

$$\psi_{ec2}(t + \tau) = e^{\frac{\tau}{2} S_1} e^{\tau S_2} e^{\frac{\tau}{2} S_1} \psi(t). \quad (83)$$

Eq. (83) indicates a way to handle multiple subsystems. For instance, if we have three subsystems $S = S_1 + S_2 + S_3$ we can first apply this expression with respect to two subsystems S_1 and $S_2 + S_3$ and get the solution with the second order accuracy expressed via $e^{\tau(S_2 + S_3)}$. We then can apply this expression to split the system $S_2 + S_3$, which leads to the following solver with the second order accuracy:

$$\psi_{ec2}(t + \tau) = e^{\frac{\tau}{2} S_1} e^{\frac{\tau}{2} S_2} e^{\tau S_3} e^{\frac{\tau}{2} S_2} e^{\frac{\tau}{2} S_1} \psi(t). \quad (84)$$

Iterative application of this splitting leads to the method modification that has the second order accuracy for N particles:

$$S = \sum_i S_i, \quad (85)$$

$$\psi_{ec2}(t + \tau) = e^{\frac{\tau}{2} S_1} e^{\frac{\tau}{2} S_2} \dots e^{\frac{\tau}{2} S_N} e^{\frac{\tau}{2} S_N} \dots e^{\frac{\tau}{2} S_2} e^{\frac{\tau}{2} S_1} \psi(t). \quad (86)$$

In practice this means that we first do a loop for updating all the particles in any random order by a half-step and then do another half-step update making a loop in the exactly reversed order. Note that the second order solver (“ec2”) requires roughly twice as much time for update as compared to the first order solver (“ec”). Nevertheless, as we demonstrate in Sec. 7.1 the increased order of accuracy can make the results much more accurate than that obtained with twice as many steps and the first order solver.

Let us also note that the presented consideration concerns only the coupling itself, while the use of estimated mid-point of the trajectory element for determining coupling coefficients introduces another error related also to the space step. For example, the described estimate becomes increasingly inaccurate if particles undergo cyclotron gyration with a poorly resolved period. The effect of this error depends on the simulated process: the case of low particle energy and strong magnetic fields can be problematic. As was mentioned, individual subcycling and correction of the initial estimated mid-point location can provide an improvement in this case.

We now turn to the consideration of two other potential improvements. The first one concerns the violation of charge conservation known to be a severe problem in some simulations. One well-known solution to this problem is Esirkepov’s charge conserving current deposition [62], which computes the current produced by the change of location for each particle in an exact agreement with the charge continuity equation. Since in our case the current is not computed, we cannot apply this method. At the same time, in our implementation there are no measures that could ensure an exact agreement with the charge continuity equation. For sufficiently large steps the errors can be accumulated, which can lead to numerical artefacts. We demonstrate the existence of this drawback in the test presented in Sec. 8. However, since we use the Fourier-based method for updating electromagnetic fields we can apply the procedure of divergence cleaning. For this we exploit the fact that exact charge conservation corresponds to maintaining the exact solution of Poisson’s equation described by eq. (69). We can thus compute charge density ρ making a loop over all particles, transform it into the bases of plane waves $\rho_{\mathbf{k}}$ using additional FFT and then correct vector $\mathbf{F}_{\mathbf{k}}$ in accordance with eq. (69). A straightforward way of such correction is to replace the projection of vector $\mathbf{F}_{\mathbf{k}}$ on vector $\mathbf{k}$ with the exact value:

$$\mathbf{F}_{\mathbf{k}} := \mathbf{F}_{\mathbf{k}} - \frac{\mathbf{k} \cdot \mathbf{F}_{\mathbf{k}}}{k} \frac{\mathbf{k}}{k} - 4\pi i \rho_{\mathbf{k}} \frac{\mathbf{k}}{k}. \quad (87)$$

Nevertheless, this procedure does not preserve energy and thus deprives our method of its characteristic property. The example presented in Sec. 8 shows that the proposed energy conservation scheme does not necessarily run into a complete conflict with divergence cleaning, but the disagreement can be essential. That is why this modification is left as an auxiliary option that does not necessarily lead to an improved accuracy of simulations.

There is however a possibility of exploiting the proposed splitting for combining the properties of both charge and energy conservation. The schematic description of this possibility is as follows. For each particle we restrict the change of electric field at the nearby nodes in accordance with charge continuity equation, quantifying the extent of change by one or several parameters (optionally using particle’s instantaneous direction of motion). Then we introduce a sub-system that couples particle’s momentum and the field change quantified by these parameters. The solution of this system can provide a way to either exactly preserve energy and charge or make a significant improvement to the method described in this paper.

The second potential improvement concerns the possibility of preserving momentum. Instead of using the part of Boris pusher related to magnetic field (or the particle momentum rotation described by eq. (58)) for solving eq. (8) we can express the corresponding part of momentum update $\Delta_{\mathbf{B}} \mathbf{p}$ via the change of electric field $\Delta \mathbf{E}$ using eqs. (29), (31):

$$\Delta_{\mathbf{B}} \mathbf{p} = \int_t^{t+\Delta t} \frac{q}{mc} \frac{\mathbf{p}}{\sqrt{1+p^2}} \times \mathbf{B} = \frac{q}{mc} \Delta \mathbf{r} \times \mathbf{B} = -\frac{V_g}{4\pi mc} \Delta \mathbf{E} \times \mathbf{B}. \quad (88)$$

Expressing the corresponding change of particles' momentum in dimensional units

$$mc\Delta_{\mathbf{B}}\mathbf{p} = -V_g \frac{c}{4\pi} \left(\sum_{j=1}^M c_j \Delta \mathbf{E} \right) \times \mathbf{B} = \sum_{j=1}^M V_g \frac{c}{4\pi} \Delta \mathbf{E}_j \times \mathbf{B}, \quad (89)$$

we see that this change of momentum is exactly equal to the change of momentum carried by the electromagnetic field. This can provide the property of momentum conservation. However, in contrast to the use of Boris pusher or eq. (58) this update does not necessarily preserve particle's energy and thus its use would deprive our method of its characteristic property. Nevertheless, this observation indicates an interesting pathway towards further developments that exploit the described splitting and particle-field coupling. One possible direction is to consider not exact but improved properties with respect to preservation of charge, energy and momentum. The other option is to consider a family of solvers that can prioritize the conservation of one or another quantity.

6. π -PIC

Here we briefly describe an open-source python library π -PIC (PI-PIC: Python-controlled Interactive PIC), which provides implementations of the described PIC solvers [67]. The developments are done within the hi- χ framework, which is an open-source collection of tools for performing simulations and data analysis in strong-field particle and plasma physics [68]. The code is designed for relativistic simulations in 3D, 2D and 1D geometry. The computational routine is based on spectral field solver and includes a basic implementation of the described energy conservation concept (mid-point CIC coupling) with the first and second order accuracy, along with the standard PIC method with Boris pusher. The inclusion of several solvers is to facilitate comparison and studies of the proposed approach. Simulations can be configured and executed using Python (or Jupyter notebook).

The code is written in C++ and resource intensive parts exploit OpenMP. For each type particles are stored in separate vectors (operated by standard library `vector`) allocated for each cell. To make possible use of multiple threads the cells are processed with a stride of 8 along x axis, which in case of CIC coupling/weighting ensures thread-safe operation (note that the estimated mid-point for a particle in a given cell can be in a neighboring cell). To implement shuffling, in the beginning of each iteration we chose a random order of offsets along x axis (from 0 to 7) and make a non-parallel loop over all the offset values. For each offset we use OpenMP to make a parallel loop over groups of cells: each group includes all the cells with the same location along x axis; the groups are shifted by the given offset and spaced apart by the value of stride along x direction. The loop over all cells along y and z (in case of 2D or 3D geometry) is made by the same thread. If a particle migrates to a neighboring cell it is thread-safe to remove it from the current vector and add to the corresponding vector of the neighboring cell (migrated processed particles are stored in the demarcated end part of the vector to avoid double processing). If a particle migrates to a distant cell, the reference to this particle is added to thread-local list of particles to be moved after the parallel loop over particles (note that in this case CIC might be insufficient and one should consider reducing time step or using more advanced coupling method). In addition, in case of energy-conserving solver the particles are shuffled in each cell to remove repeated effect of ordered interaction with the field (for more details see Sec. 7.2). The implementation of the energy-conserving solver with the second order accuracy is based on making two identical loops with the exactly reversed order (to maintain the exactly reversed order the particles are not migrated during the first loop). This was measured to increase the computational demands by about a factor of two.

π -PIC is developed as a Python library, which offers a possibility to configure data containers for fields and particles. All data elements can be accessed for output and advanced in time using one of the following solvers (one can also assign external callbacks (extensions) developed in Python, C++ or other languages to be applied during particle loops):

- “ec” stands for the described above simplest version of energy-conserving solver with the first order accuracy and coupling coefficients determined by the CIC weighting for the predicted mid-point location;
- “fourier_boris” stands for the standard Boris pusher with CIC weighting combined with the Fourier-based solver of Maxwell's equations, divergence cleaning and time-centered application of current, i.e. half-step effect before and after field update;
- “ec2” stands for the variant of energy-conserving solver that has the second order accuracy reached by making two loops over particles with mutually reversed order of processing (see sec. 5).

In addition one can disable divergence cleaning for the Boris method using the command “fourier_solver_settings (divergence_cleaning = False)”. In what follows this option is labeled “boris_no_div_cleaning”.

Although the code optimization is beyond the scope of this paper, we here report some basic measurements and observations for future reference. We consider the simulation of laser-solid interaction (see Sec. 8 for description and Appendix C for the Python code), using 2D grid with 100 particles per cell in the plasma bulk and the rate of particle migration between cells ranging from 0.06 to 0.2 (0.5 in case of Boris pusher due to numerical heating). When using 8 cores of Intel Core i7-10700K@3.8 GHz the average update time per particle ranges from 12 to 14 ns for `fourier_boris` solver, while for `ec` and `ec2` it ranges from 10 to 11 and from 19 to 20 ns, respectively. The update time with `ec` appears to be even shorter than that with `fourier_boris`, which can be due to additional memory calls for updating grid values of charge density and current in the latter case. Compared to these time costs, shuffling particles do not take a significant amount of time, probably due to good use of caching.

Computations related to spectral field solver are also arranged using OpenMP loop over individual 1D FFT calls. The time demands should be attributed to the used FFTW library [69]. Nevertheless we note that the described method doesn't require FFT for the current. For reference, for the above mentioned CPU the best time for $64 \times 64 \times 64$ was measured to be 10 ns per cell.

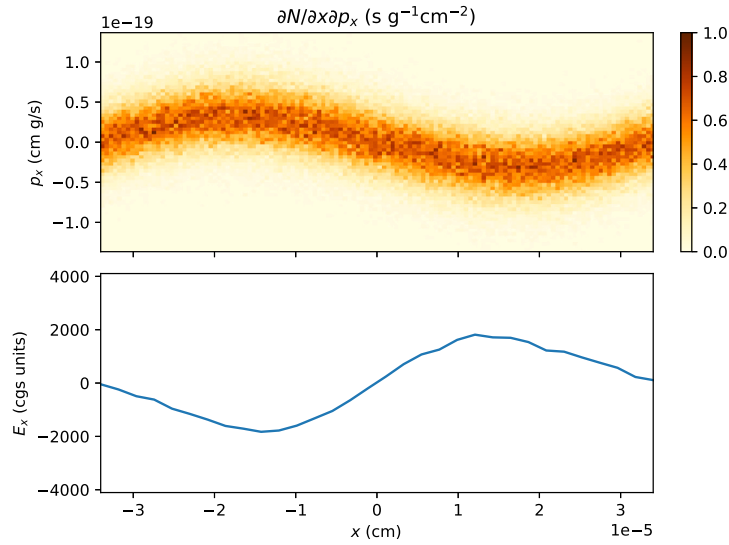


Fig. 2. The 25-th image created by the example presented in subsection 6.1. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

We should emphasize that the described strategy for parallel processing is significantly restricted as compared to the possibilities for the conventional PIC method and might be inefficient for a large number of cores and especially for GPU. The parallel implementation for distributed memory of supercomputers can be done using decomposition into domains processed by each supercomputer node. However, in contrast to conventional PIC computations the proposed algorithm would require transferring electric field values at the bordering nodes for alternating processing of cells at the interfaces. That is why the order of offsets must be synchronized. The implementation of the algorithm with the second order accuracy would require making an additional loop with the inversed order of processing.

The controls of π -PIC are rather minimalistic, yet they can be found sufficient for many cases. To maintain reasonable performance the communication between the data of simulations and Python is arranged using C callbacks provided by Numba package. This approach has been previously developed and used in hi- χ framework [70]. From Python it is possible to call a full cycle over all cells or particles so that for each particle or cell a function defined in Python is called. Along with the address of that function, it is possible to pass pointers to arbitrary arrays of integer or double type to be used or amended during the function calls. The use is exemplified in the next subsection.

The interface is based on several assumptions:

- CGS units are used for all dimensional variables;
- the components of vectors are enumerated: x , y , z correspond to [0], [1], [2], respectively;
- the sizes of the grid n_x , n_y and n_z should be powers of two to facilitate FFT;
- 2D simulation is enabled by setting $n_z=1$, 1D simulations are enabled by $n_z=1$ and $n_y=1$;
- the space is assumed to be periodic (other geometries can be implemented within Python).

6.1. Example: plasma oscillation

In Appendix A we provide a Python file that illustrates the use of π -PIC (see [67] for more examples and configuration files used for simulations in the manuscript). The case in question is the process of plasma oscillation. The code produces a sequence of images. The 25-th image is shown in Fig. 2. When choosing solver it is possible to select `fourier_boris`, `ec` or `ec2`.

6.2. Landau damping

In this subsection we verify the correctness of all the implemented solvers by simulating Landau damping in cases of weak and strong perturbation. We consider parameters of the standard cases described in Ref. [71]. We consider non-relativistic temperature $T = (2/3)10^{-6}mc^2$ and set the initial plasma density to $N(x) = 1 + \alpha \sin(2\pi x/(X_{\text{Max}} - X_{\text{Min}}))$, where the limits of the simulation region are $X_{\text{Max}} = -X_{\text{Min}} = 2\pi\lambda_D$ with λ_D being the Debye length. The initial electric field is defined from the assumption of uniform positive background of ions. We use one-dimensional computational region with $N_x = 16$ cells and 10^4 particles per cell. The time step is $\Delta t = T_p/64$. The results of the simulations and their comparison with theoretical rates for $\alpha = 0.01$ and $\alpha = 0.5$ are presented in Fig. 3. Variation of computational parameters does not show any notable benefit of using energy conserving method in this problem. This is natural because the time and space steps needed to resolve the physics in question is much less the plasma period and Debye length. In this case the conventional Boris method is known to be stable and the numerical heating due to minor violation of energy conservation is happened to be negligible as compared to the scales of interest.

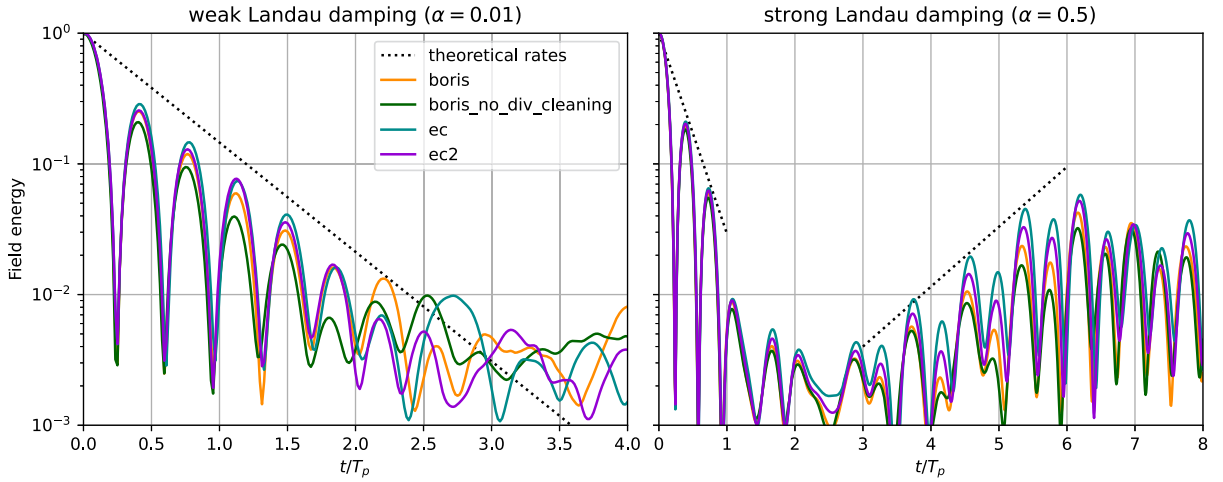


Fig. 3. Verification of the developed solvers based on the simulation of Landau damping in cases described in Ref. [71]. Label “boris_no_div_cleaning” stands for Boris pusher with disabled divergence cleaning. The plots show temporal evolution of the energy of electric field normalized to the initial value. The slopes of theoretical rates (dotted lines) are taken from Ref. [71].

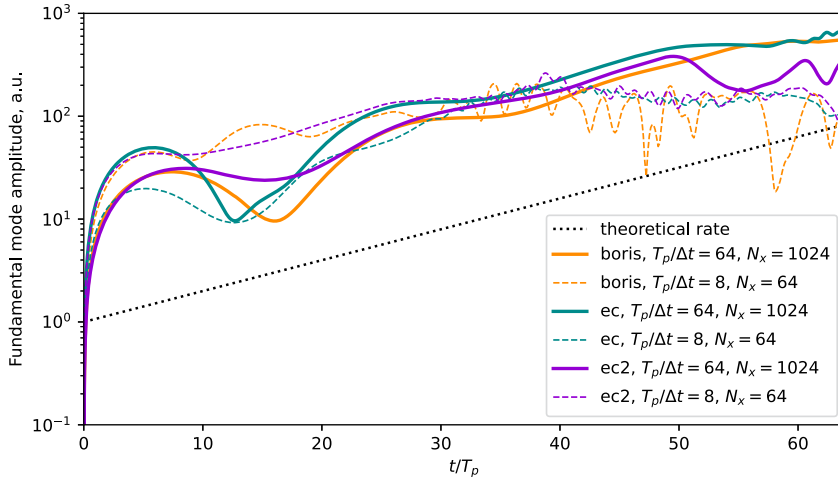


Fig. 4. Verification of the developed solvers in relativistic case based on the simulation of two-stream instability.

6.3. Relativistic two-stream instability

In order to verify the correctness of the solvers in relativistic case we simulate the relativistic two-stream instability and compare the results with the rate given in, e.g. Ref. [72]. We consider two counter-propagating streams of electrons with equal density $N = 1/2$ and gamma factor being equal to $\gamma_s = 10$ for both streams. The initial momentum is modified by a factor $1 \pm 0.01 \sin(2\pi x/(X_{\text{Max}} - X_{\text{Min}}))$ to set a more stable seed for the instability. Following the theory consideration presented in [72] the limits of computational regions $X_{\text{Max}} - X_{\text{Min}} = \lambda_0 \approx 1.6329 \gamma_s^{2/3} v_s T_p$ are set to include a single mode with maximal growth rate $\Gamma \approx 0.011(2\pi/T_p)$ (v_s is the velocity that corresponds to gamma factor γ_s). We again use 10^4 particles per cell, but consider two options for the time and space step, the latter being defined via the number of grid points $\Delta x = (X_{\text{Max}} - X_{\text{Min}})/N_x$: $\Delta t = T_p/64$, $N_x = 1024$ (high resolution) and $\Delta t = T_p/8$, $N_x = 64$ (low resolution). For comparison we plot the amplitude of the fundamental mode as a function of time in logarithmic scale together with a line with the slope defined by the theoretical value of the growth rate Γ . The results for different solvers are compared in Fig. 4. The capabilities of all the solvers to deal with the case of low resolution are similar. The only minor deviation worth mentioning is the rapid field amplitude oscillations observed for the “boris” solver at the late, highly non-linear stage of the simulated process.

7. Basic properties

7.1. The limit of low resolution

Even though the method warrants energy conservation by construction, we need to verify that its use is not precluded by numerical artefacts that do not violate this conservation law. For example, preserving energy per se does not contradict to the

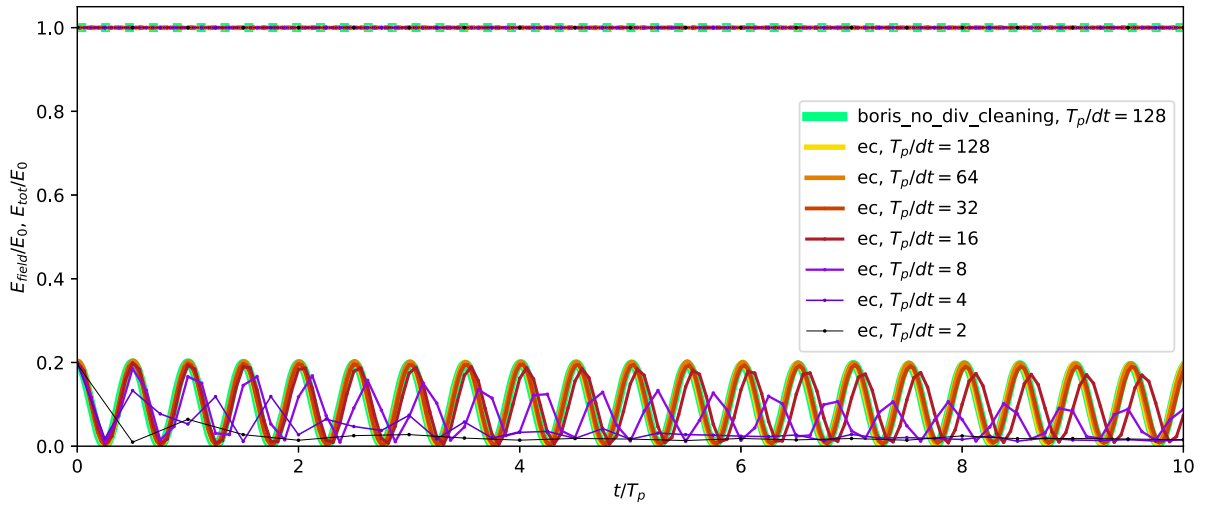


Fig. 5. Analysis of the effect of low space and time resolution on an oscillating cold plasma simulations carried out with the energy conserving solver with first order accuracy (labeled “ec”). The plot shows the dependencies of electromagnetic E_{field} (solid curves) and total E_{tot} (dotted curves) energy on time t during 10 plasma periods T_p for difference values of time step dt . The values of energy are normalized to the initial energy E_0 . The results are compared with high-resolution simulation carried out with Boris pusher coupled with the Fourier-based solver for Maxwell’s equations (see details in the text) with disabled divergence cleaning (labeled as “boris_no_div_cleaning”).

hypothetical process of transferring kinetic energy of thermal particle motion to electromagnetic field of plasma oscillations, i.e. to the process of spontaneous generation of plasma oscillations. (Clearly, such a behavior would contradict to the second law of thermodynamics; any plasma oscillations should in opposite decay in line with Landau damping.) To assess the presence of this and potentially other numerical artefacts in case of low resolution we perform a more extensive study of plasma oscillations as probably one of the most indicative plasma processes.

We consider a simulation of 10 plasma periods for a setup described in Sec. 6.1 with $X_{Min}, X_{Max} = -612.4 * DebyeLength, 612.4 * DebyeLength$ represented by $n_x = 32$ cells, using 100 particles per cell and varied time step (see Appendix B). To test the applicability limits of the developed solvers we intentionally consider low resolution for both time and space. The results for different time resolution are presented in Fig. 5.

In case of using the energy-conserving method (labeled “ec”) the total energy is preserved to within machine accuracy (deviation is below 10^{-11}) independently of the time step. As a benchmark we show the result of a simulation with high temporal resolution using the standard Boris pusher in combination with spectral field solver with disabled divergence cleaning. As one can see, for high resolution the results of both solvers are in a good agreement.

Let us consider how the results change when resolution becomes low. We can see that in the limit of low resolution the presented energy-conserving method doesn’t cause any more severe deviations than increased rate of decay and frequency altering, both effects being fairly minor until about 16 steps per plasma period. In this case, the increased rate of decay becomes significant for the resolution of 8 steps per plasma period, while even for 4 steps per period the effect of plasma oscillations is qualitatively reproduced. In case of 2 steps per plasma period the oscillations decay almost immediately, while still not causing any clearly destructive nonphysical plasma states.

These results indicate that the “ec” method can be found capable of performing simulations with a fairly large time step without causing any destructive numerical artefacts. This means that even if the time step is insufficient for resolving plasma oscillations in some regions of high plasma density, these regions are expected to remain physically stable. The latter can be useful if the physics of interest doesn’t require good resolution of many plasma oscillations, or such oscillations are resolved using reduced time step during a limited part of simulation.

To identify the applicability limitations of all the developed solvers we extend the duration of simulation to 100 plasma oscillations and consider the results for a sufficiently low resolution to cause notable deviation from the exact solution. The results are presented in Fig. 6. The most accurate computations are achieved with “ec2” solver and thus in this case we use its result with high resolution ($T_p/dt = 128$) as a benchmark. One can see that the values obtained with the Boris pusher without divergence cleaning (labeled “boris_no_div_cleaning”) and $T_p/dt = 8$ fairly accurately follow the benchmark values up until $t \approx 80T_p$ and then run into an instability. This instability is caused by accelerated accumulation of errors due to inaccurate current deposition and can be suppressed by divergence cleaning, which is well demonstrated by the result obtained with the same time step by the Boris pushed with enabled (by default) divergence cleaning (labeled “boris”). Nevertheless, one can see that the stability of the “boris” solver is achieved at the cost of notable continuous numerical heating that completely alters the structure of the field energy oscillations starting from about $t = 20T_p$. Note, however, that such a rapid numerical heating is caused by an extremely low temporal and spatial resolution $\Delta x \approx 30\lambda_D$, where λ_D is the Debye length.

From Fig. 6 we can also see that “ec2” solver provides a remarkably accurate result, which almost perfectly agrees with the benchmark (the deviation is rather caused by aliasing related to the fact of having just 4 points per each oscillation of field energy).

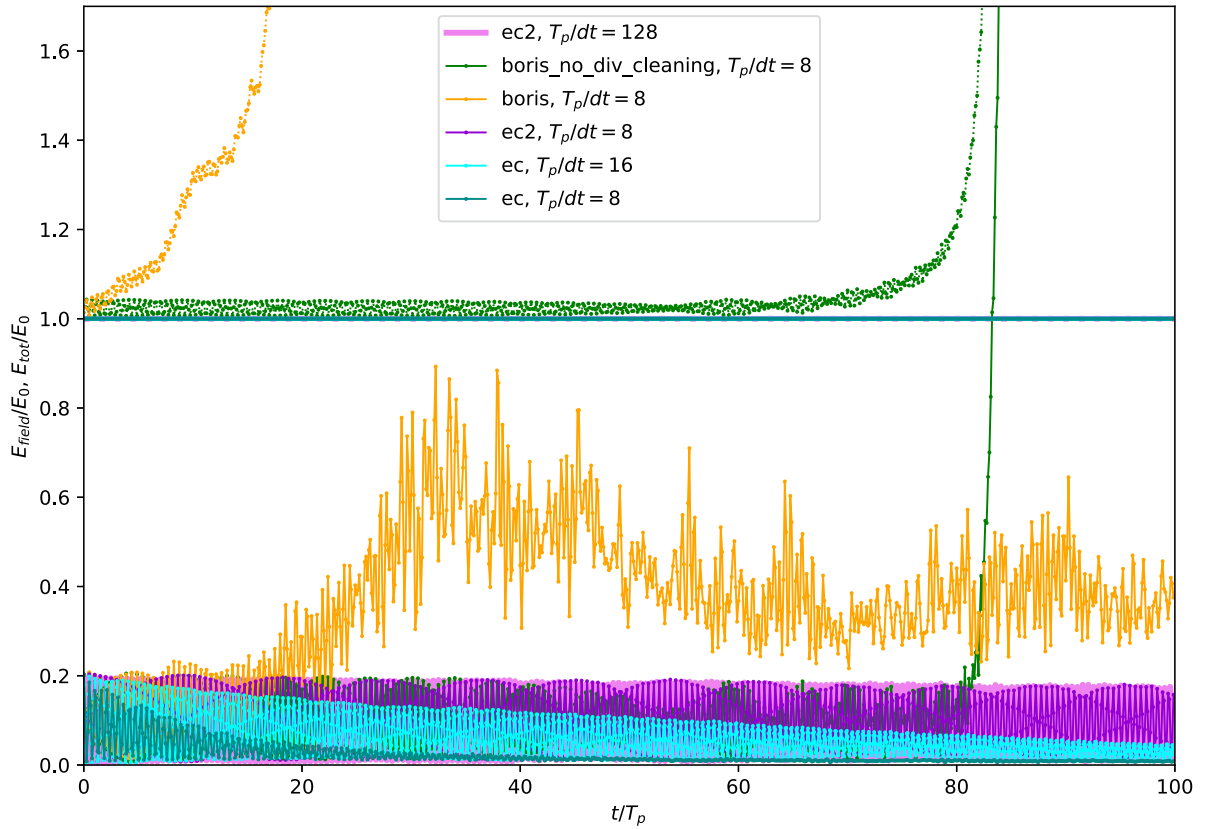


Fig. 6. The comparison of the capabilities of different solvers shown with the same data as in Fig. 5 but for a time interval of $100T_p$ and a sufficiently low space and time resolution to make visible the limitations. The case labeled “boris” stands for the Boris pusher with enabled divergence cleaning.

At least the results are significantly more accurate than that obtained with “ec” solver with factor 2 smaller time step. Given that “ec2” involves two loops over particles per iteration, we can clearly see the benefit of achieving the second order accuracy.

7.2. The cost of using low resolution

The results presented in the previous subsection indicate that in the limit of low resolution the use of energy-conserving method leads to an increased rate of decay for plasma oscillations. It is interesting to consider this effect in more detail. This is to reveal potentially less prominent effects that constitute the price we pay for using low resolution. For this purpose in this subsection we consider how a low resolution affects the particle distribution in the phase space.

Since in π -PIC the particles are stored and processed in a cell-by-cell order, the effect of order mentioned in the end of Sec. 3 can clearly play a significant role. In case of single loop over all particles, the particles that arrive to a given cell are processed either before or after all other particles in this cell. Since the chance of migration is correlated with energy, this clearly introduces a bias. That is why when energy-conserving scheme is used in π -PIC the particles are shuffled in each cell (before each iteration) and processed in two halves: first we make a loop over all cells processing a half of particles and then make another loop over all cells processing the remaining particles. To see the effect of such shuffling, we present a separate result for simulation without it.

To perform analysis we repeat the simulation of a single plasma oscillation described in sec. 6.1 with $n_x=8$. In Fig. 7 we show the phase space $x-p_x$ described in Fig. 2 for different time step values and for different setups: Boris pusher, energy-conserving method without shuffling and energy-conserving method with shuffling. We can see that shuffling indeed significantly suppresses the effect of order and overall improves the accuracy of simulation in the limit of low resolution. In case of low resolution the energy-conserving method with shuffling shows a rather reasonable distribution that doesn’t seem to deviate dramatically from the thermal distribution, which one could observe after the decay of plasma oscillations.

8. Verification and capabilities

In this section we pursue two goals. First, we verify the correctness of the π -PIC code. Second, we show that in a rather generic case the proposed energy-conserving method indeed makes the code capable of performing simulations with time and space steps notably larger than that needed to resolve plasma period and Debye length, respectively.

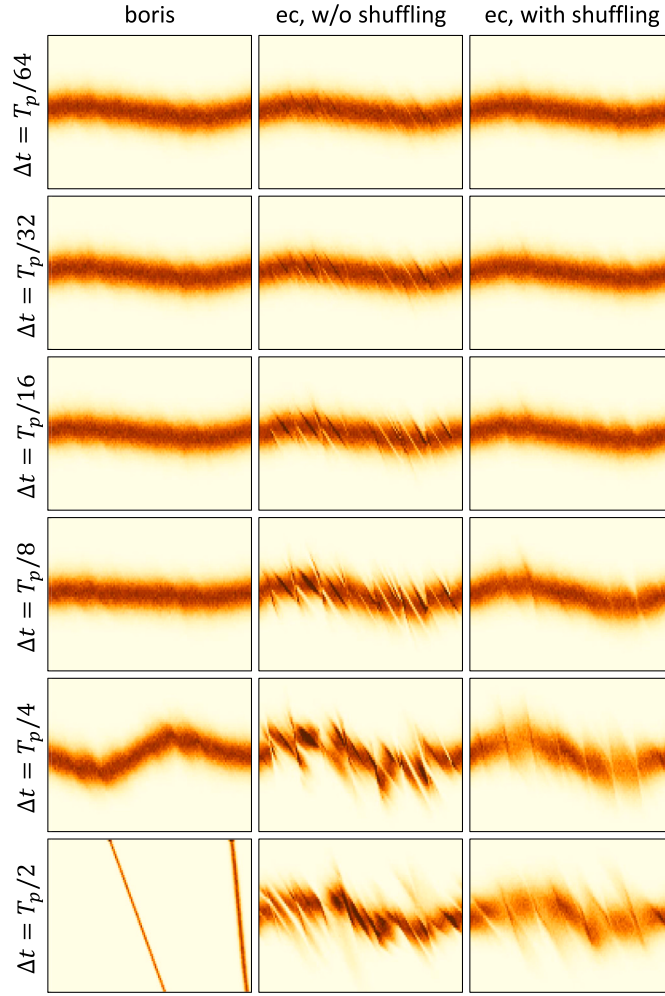


Fig. 7. Distributions of particles in the space of x - p_x (for the description of axes and the color scheme see Fig. 2) for different setups: Boris pusher with spectral field solver (left column), energy-conserving method without shuffling (central column), and energy-conserving method with shuffling (right column).

The verification and comparison can be more indicative if the simulated process involves complex relativistic plasma dynamics. In this case exact analytical results that provide complex data for comparison are rarely achievable. One example is the theory for relativistic radiation reflection from dense plasmas, referred to as Relativistic Electronic Spring (RES) [73,74]. The RES theory provides a way to compute the field structure of the electromagnetic pulse after reflection from a plasma surface with arbitrary density distribution. The results become increasingly more accurate for higher intensity of the incident pulse so that highly relativistic plasma dynamics is induced. In case of non-zero incidence angle this process can still be simulated in 1D geometry using a moving reference frame (in π -PIC this can be arranged by setting initial moment for all particles). Nevertheless, we intentionally consider the process in laboratory reference frame to make testing more comprehensive. We choose to perform verification and comparison using 2D simulations (3D case can be considered by setting $n_z \neq 1$ and modifying the shape of the pulse along z).

We consider a short circularly polarized pulse with the wavelength of $\lambda = 1 \mu\text{m}$ impinging on a plasma layer with the incidence angle of $\theta = \pi/3$. The layer has smooth density drops within $1 \mu\text{m}$ and the shape of $\propto \sin^2$. The bulk density is $N_0 = 100n_{cr}$, where $n_{cr} = \pi mc^2 / e^2 \lambda^2$ is the plasma critical density. The peak field amplitude of the pulse is $100a_0$, where $a_0 = 2\pi mc^2 / \lambda |e|$ is the relativistic field amplitude. We perform simulations for different time and space steps keeping $2c\Delta t = \Delta x = \Delta y$. The initial plasma temperature is chosen so that the Debye length is set to $\lambda_D = \lambda/640$. In all simulations we use 30 particles per cell for the inner part of the plasma region. Other details can be read from the listing of the Python code presented in Appendix C. Note that this setup is related to experiments on the interaction of intense laser pulses with solid targets, which is motivated by various applications including high harmonic generation and particle acceleration.

In Fig. 8 we show the initial state of the field and plasma. The pulse, shown via B_z field component, propagates toward point $x = y = 0$. The state of the system at the time instance $t \approx 3.5 \times 10^{-14}$ s is shown in Fig. 9 for all considered cases. One can see the outgoing pulse that can be benchmarked against the RES results. In Fig. 10 we compare the transverse field along the line $x = y \tan \theta$ at $t \approx 3.5 \times 10^{-14}$ s.

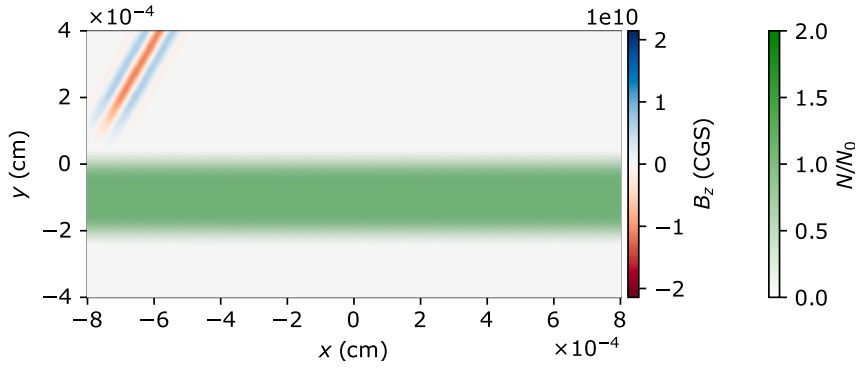


Fig. 8. Field B_z component and plasma density N normalized to $N_0 = 100n_{cr}$ in the initial moment of simulations. The pulse propagates towards point $x = y = 0$.

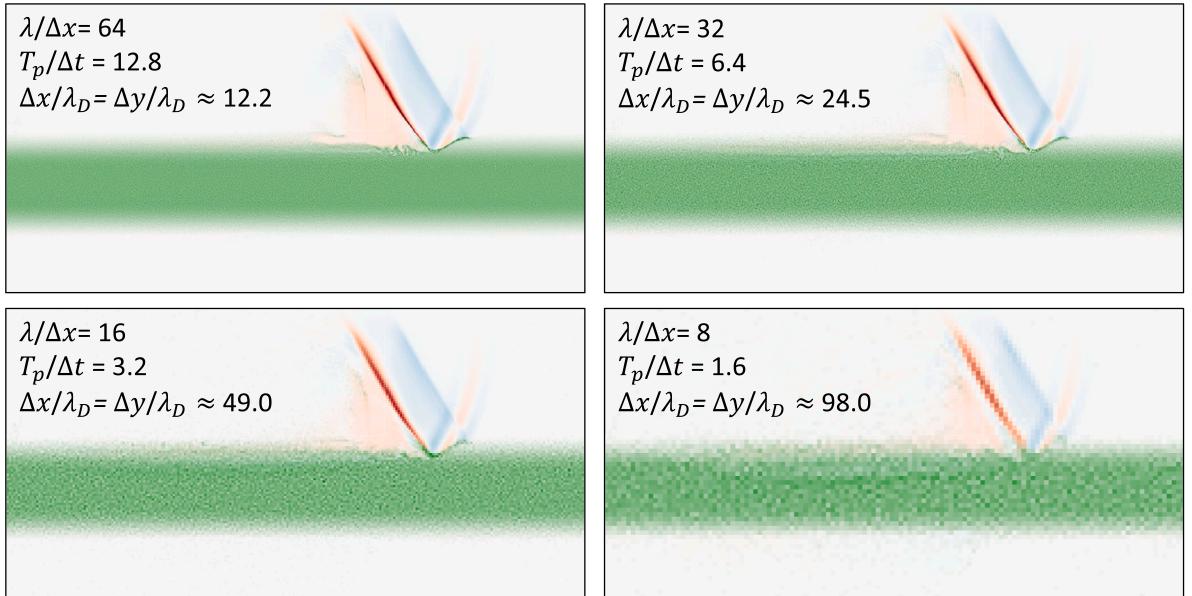


Fig. 9. Results of simulations for different time and space steps. For the description of axes and color maps see Fig. 8.

From Fig. 10 we can see that numerical simulations result in shapes that are close to what obtained with the RES theory. This indicates that the code is likely to work correctly. Similar results are obtained for all the developed solvers. Certainly the computation based on the RES theory provides only an approximate result that is useful for verifying the correctness of implementation, but not for comparing the capabilities of the solvers. We should not expect the exact agreement with the RES theory because of many reasons including limited transverse size and limited amplitude of the pulse (the results obtained with the RES theory become increasingly accurate with the increase of incident radiation intensity). We can also see that the results remain fairly accurate even in case the time step is comparable with the plasma period $T_p = \lambda/10c$ for the density inside the target. Note that in the case of $T_p/\Delta t = 1.6$, it is likely that the low spatial resolution of the wavelength λ affects the accuracy of the simulation.

In the context of this numerical experiment one characteristic property of the proposed method is the absence of numerical heating, which is a common consequence of imperfect energy conservation in case of insufficient time and space resolution for conventional schemes. To assess a potential benefit we consider the process of plasma heating by the incident electromagnetic pulse. In Fig. 11 we plot the kinetic energy of electrons after the interaction at $t = 70$ fs normalized to the initial value at $t = 0$ fs as a function of resolution for different numerical schemes. As previously, the “boris” scheme here denotes a combination of using Boris pusher with CIC weighting in combination with the dispersion-free Fourier solver of Maxwell’s equations and divergence cleaning for charge conservation. The increased kinetic energy observed for the Boris pusher is related to numerical heating, while for energy conserving methods the increased energy at small resolution can be associated with overestimation of energy coupling which can naturally occur due to order-of-one-step deeper penetration of electromagnetic radiation behind a sharp radiation-plasma interface. One can see that as compared to the results of “boris” scheme, energy conserving schemes achieve similar accuracy levels with factor 2-4 larger space and time steps, which can be roughly translated to a speedup factor 8 – 64 for 2D case and factor 16 – 256 for 3D case. This indicates that the proposed method can be useful for long-term simulations that require absence of numerical heating. One possible application is the studies of laser-driven ion acceleration by the mechanism of Target Normal Sheath Acceleration [75]. The

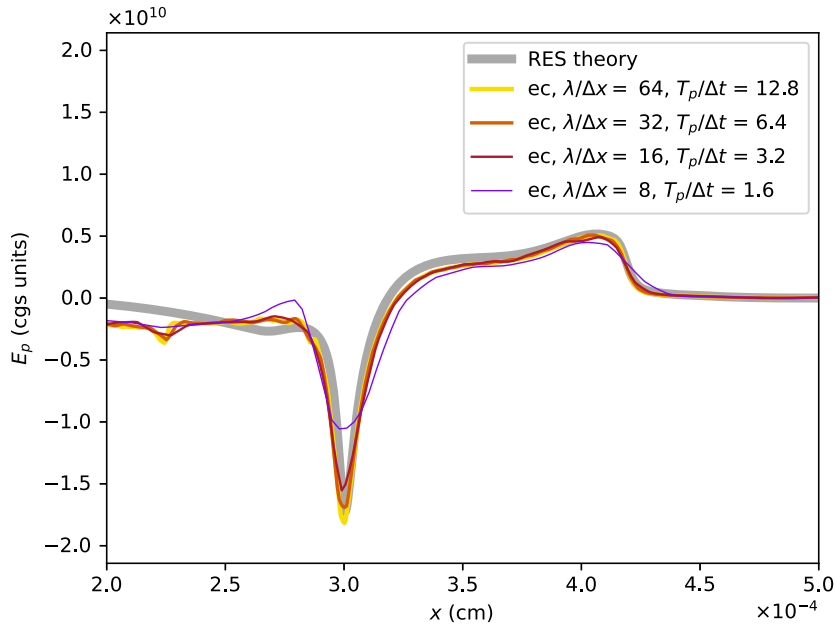


Fig. 10. Comparison of the transverse electric field obtained using the RES theory and simulations with different time and space steps.

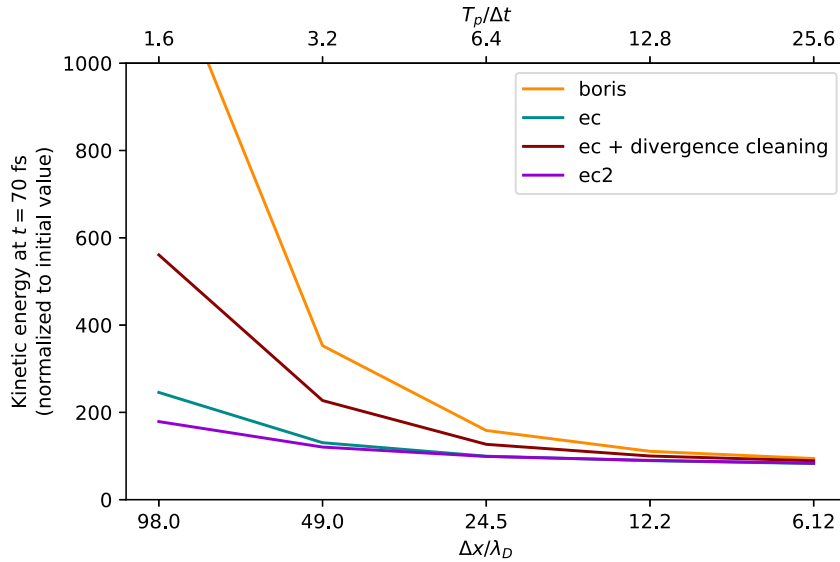


Fig. 11. Comparison of the capabilities of different solvers for simulating plasma heating in the process of intense electromagnetic pulse interaction with overdense plasma. The total kinetic energy after the interaction (at $t = 70$ fs) is normalized to the initial value and plotted as a function of numerical resolution for different solvers.

method or its improved versions can be also found useful for suppressing numerical instabilities in simulations of laser wake-field acceleration [76] in a boosted reference frame [77].

Although the density distribution and field states shown in Fig. 9 look almost identical for all the solvers, let us make a more detailed consideration of the differences between the solvers. First, we can note that the combination of energy conserving solver with divergence cleaning (labeled “ec + divergence cleaning”) has no property of exact energy conservation, giving an origin for numerical heating, which however is slightly weaker than that in the case of “boris” solver. This indirectly indicates that in case of low resolution the energy conservation appears to be in an increasingly notable disagreement with charge conservation (otherwise divergence cleaning would not have any effect). To see this effect, we plot the state of system at $t = 35$ fs for different solvers in Fig. 12. In case of “ec” behind the sliding reflection point one can note a trace of rapid density drop, which does not oscillate and in the absence of moving ions can clearly be associated with accumulated deviation from the charge conservation. Indeed, this trace is suppressed outside interaction region in case of applying divergence cleaning to the “ec” solver. This indicates that imposing charge

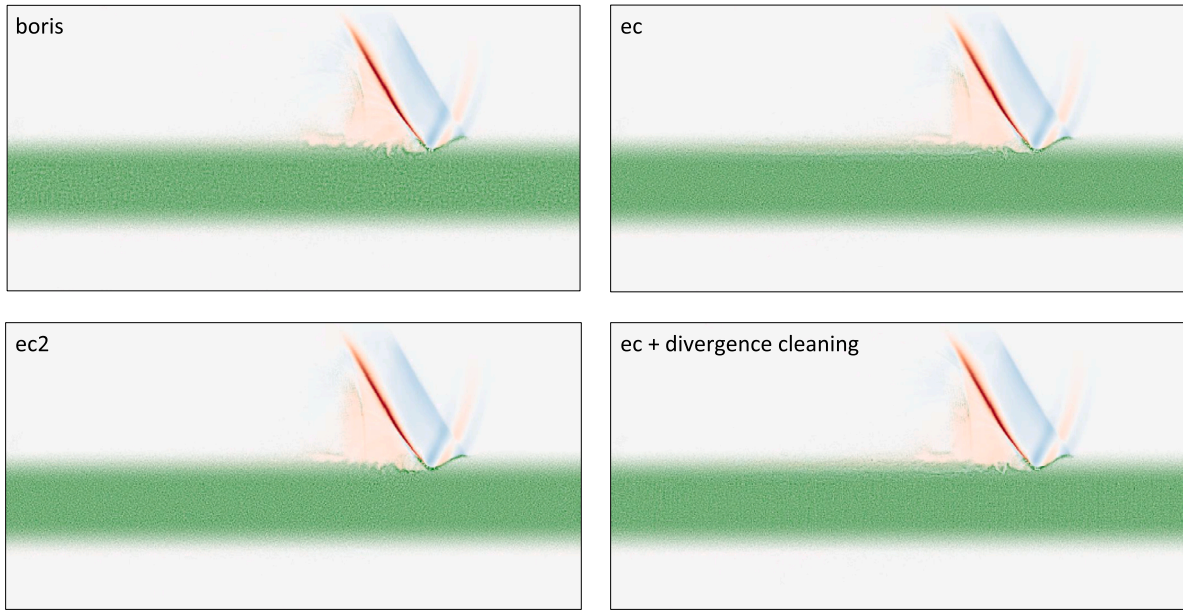


Fig. 12. Comparison of the system state at $t = 35$ fs obtained with different solvers. The axes are the same as in Fig. 8.

conservation within the proposed approach is likely to be extremely useful and can significantly improve accuracy. The second order energy conserving method (“ec2”) is likely to suffer from imperfect charge conservation but the higher accuracy seems to make it much weaker, so that it is not clearly visible in Fig. 12.

Although the results indicate a potential usefulness of the proposed scheme, we should admit that we here do not consider many possible improvements to the standard scheme, including higher order form factors for particles, current smoothing and/or high harmonic filtering (see e.g. [15,78]). Note however that in contrast to the presented method, the use of these improvements can imply an effective reduction of resolution, increased computational demands for larger number of distant memory calls, as well as a continuous energy dissipation and/or artificial non-local field modification. At the same time, as described above, there are several possible improvements to the proposed scheme, including imposed charge and momentum conservation. A more conclusive analysis requires further developments and studies with more elaborate measures of accuracy.

9. Conclusions

In this paper we described a way to modify the standard relativistic PIC method so that the exact energy conservation is enforced, while the computational scheme remains explicit and involves similar amount of computations and memory calls. We derived governing principles and a basic algorithm for their implementation. We considered possible limitations and some ways for overcoming them. To facilitate further studies and use of the proposed method we developed and made publicly available the π -PIC code, which is relativistic energy-conserving PIC code based on spectral field solver [67]. The performed tests indicate that the code and the proposed approach in general can be found useful for plasma simulations.

CRediT authorship contribution statement

Arkady Gonoskov: Conceptualization, Data curation, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Software, Validation, Visualization, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgements

The work is supported by the Swedish Research Council (Grants No. 2017-05148 and 2019-02376). The author would like to thank Joel Magnusson for the help with π -PIC interfaces and deployment.

Appendix A. Python code used for Sec. 6.1

```

1  import pipic
2  from pipic import consts, types
3  import matplotlib.pyplot as plt
4  import numpy as np
5  from numba import cfunc, carray
6  import os, time
7  from pipic.consts import electron_mass, electron_charge, light_velocity
8
9  # =====SIMULATION INITIALIZATION=====
10 temperature = 1e-6*electron_mass*light_velocity**2
11 density = 1e+18
12 debye_length = np.sqrt(temperature/(4*np.pi*density*electron_charge**2))
13 plasma_period = np.sqrt(np.pi*electron_mass/(density*electron_charge**2))
14 l = 128*debye_length
15 xmin, xmax = -1/2, 1/2
16 field_amplitude = 0.01*4*np.pi*(xmax-xmin)*(-electron_charge)*density
17 nx = 128
18 time_step = plasma_period/64
19
20 # -----setting solver and simulation region-----
21 sim = pipic.init(solver='ec', nx=nx, xmin=xmin, xmax=xmax)
22
23 # -----adding electrons-----
24 @cfunc(types.add_particles_callback)
25 def density_callback(r, data_double, data_int):
26     return density
27
28 sim.add_particles(name='electron', number=1000*nx,
29                 charge=electron_charge, mass=electron_mass,
30                 temperature=temperature, density=density_callback.address)
31
32 # -----setting initial field-----
33 @cfunc(types.field_loop_callback)
34 def setField_callback(ind, r, E, B, data_double, data_int):
35     E[0] = field_amplitude*np.sin(2*np.pi*r[0]/(xmax-xmin))
36
37 sim.field_loop(handler=setField_callback.address)
38
39
40 # =====OUTPUT=====
41 fig, axs = plt.subplots(2, constrained_layout=True)
42
43 # -----preparing output for electron distribution f(x, px)-----
44 xpx_dist = np.zeros((64, 128), dtype=np.double)
45 pxLim = 5*np.sqrt(temperature*electron_mass)
46 inv_dx_dpx = (xpx_dist.shape[1]/(xmax-xmin))*(xpx_dist.shape[0]/(2*pxLim))
47
48 @cfunc(types.particle_loop_callback)
49 def xpx_callback(r, p, w, id, data_double, data_int):
50     ix = int(xpx_dist.shape[1]*(r[0]-xmin)/(xmax-xmin))
51     iy = int(xpx_dist.shape[0]*0.5*(1+p[0]/pxLim))
52     data = carray(data_double, xpx_dist.shape, dtype=np.double)
53     if iy >= 0 and iy < xpx_dist.shape[0]:
54         data[iy, ix] += w[0]*inv_dx_dpx/(3*density/pxLim)
55
56 axs[0].set_title('$\partial N / \partial x \partial p_x$ (s g$^{-1}$cm$^{-2}$)')
57 axs[0].set_ylabel='$p_x$ (cm g/s)')
58 axs[0].xaxis.set_ticklabels([])
59 plot0 = axs[0].imshow(xpx_dist, vmin=0, vmax=1,
60                      extent=[xmin, xmax, -pxLim, pxLim], interpolation='none',
61                      aspect='auto', cmap='YlOrBr')
62 fig.colorbar(plot0, ax=axs[0], location='right')

```

```

63
64 def plot_xpx():
65     xpx_dist.fill(0)
66     sim.particle_loop(name='electron', handler=xpx_callback.address,
67                       data_double=pipic.addressof(xpx_dist))
68     plot0.set_data(xpx_dist)
69
70 # -----preparing output of Ex(x)-----
71 Ex = np.zeros((32,), dtype=np.double)
72
73 @cfunc(types.it2r_callback)
74 def Ex_it2r(it, r, data_double, data_int):
75     r[0] = xmin+(it[0]+0.5)*(xmax-xmin)/Ex.shape[0]
76
77 @cfunc(types.field2data_callback)
78 def get_Ex(it, r, E, B, data_double, data_int):
79     data_double[it[0]] = E[0]
80
81 axs[1].set_xlim([xmin, xmax])
82 axs[1].set_ylim([-field_amplitude, field_amplitude])
83 axs[1].set(xlabel='$x$ (cm)', ylabel='$E_x$ (cgs units)')
84 x_axis = np.linspace(xmin, xmax, Ex.shape[0])
85 plot_Ex_ = axs[1].plot(x_axis, Ex)
86
87 def plot_Ex():
88     sim.custom_field_loop(number_of_iterations=Ex.shape[0], it2r=Ex_it2r.address,
89                           field2data=get_Ex.address, data_double=pipic.addressof(Ex))
90     plot_Ex_.set_ydata(Ex)
91
92
93 # =====SIMULATION=====
94 output_folder = 'basic_example_output'
95 if not os.path.exists(output_folder):
96     os.makedirs(output_folder)
97 time_start = time.time()
98 for i in range(32):
99     sim.advance(time_step=time_step, number_of_iterations=2)
100     plot_xpx()
101     plot_Ex()
102     fig.savefig(output_folder + '/im' + str(i) + '.png')
103     if i == 25:
104         fig.savefig(output_folder + '/fig2.pdf')
105     print(i, '/', 32)

```

Appendix B. Python code used for Sec. 7.1

```

1 import pipic
2 from pipic import consts, types
3 import matplotlib.pyplot as plt
4 import numpy as np
5 from numba import cfunc, carray
6 from pipic.consts import electron_mass, electron_charge, light_velocity
7
8
9 #=====SIMULATION INITIALIZATION=====
10 temperature = (2/3) * 1e-6 * electron_mass * light_velocity**2
11 density = 1e+18
12 debye_length = np.sqrt(temperature / (4*np.pi * density * electron_charge**2))
13 plasma_period = np.sqrt(np.pi * electron_mass / (density * electron_charge**2))
14 xmin, xmax = -612.4*debye_length, 612.4*debye_length
15 field_amplitude = 0.001 * 4*np.pi * (xmax - xmin) * electron_charge * density
16 nx = 32 # number of cells
17 iterations_period = 8

```

```

18 time_step = plasma_period/iterations_period
19 fig_label = 'ec2'
20
21 #-----setting solver and simulation region-----
22 if fig_label == 'boris':
23     sim = pipic.init(solver='fourier_boris', nx=nx, xmin=xmin, xmax=xmax)
24 if fig_label == 'boris_no_div_cleaning':
25     sim = pipic.init(solver='fourier_boris', nx=nx, xmin=xmin, xmax=xmax)
26     sim.fourier_solver_settings(divergence_cleaning=False)
27 if fig_label == 'ec':
28     sim = pipic.init(solver='ec', nx=nx, xmin=xmin, xmax=xmax)
29 if fig_label == 'ec2':
30     sim = pipic.init(solver='ec2', nx=nx, xmin=xmin, xmax=xmax)
31
32 #-----adding electrons-----
33 @cfunc(types.add_particles_callback)
34 def density_callback(r, data_double, data_int):# callback function
35     return density # can be any function of coordinate r[0]
36 sim.add_particles(name='electron', number=nx*100,
37                 charge=-electron_charge, mass=electron_mass,
38                 temperature=temperature, density=density_callback.address)
39
40 #-----setting initial field-----
41 shift = np.pi/nx # a shift to mitigate aliasing
42
43 @cfunc(types.field_loop_callback)
44 def setField_callback(ind, r, E, B, data_double, data_int):
45     E[0] = field_amplitude*np.sin(2*np.pi*r[0]/(xmax - xmin) + shift)
46
47 sim.field_loop(handler=setField_callback.address)
48
49
50 #=====OUTPUT=====
51 fig, axs = plt.subplots(2, constrained_layout=True)
52
53 #-----preparing output for electron distrribution f(x, px)-----
54 xpx_dist = np.zeros((64, 128), dtype=np.double)
55 pxLim = 5*np.sqrt(temperature * electron_mass)
56 inv_dx_dpx = (xpx_dist.shape[1]/(xmax - xmin))*(xpx_dist.shape[0]/(2*pxLim))
57
58 @cfunc(types.particle_loop_callback)
59 def xpx_callback(r, p, w, id, data_double, data_int):
60     ix = int(xpx_dist.shape[1]*(r[0] - xmin)/(xmax - xmin))
61     iy = int(xpx_dist.shape[0]*0.5*(1 + p[0]/pxLim))
62     data = carray(data_double, xpx_dist.shape, dtype=np.double)
63     if iy >= 0 and iy < xpx_dist.shape[0]:
64         data[iy, ix] += w[0]*inv_dx_dpx
65
66 axs[0].set_title('$\partial N / \partial x \partial p_x$ (s g$^{-1}$cm$^{-2}$s)')
67 axs[0].set_ylabel='$p_x$ (cm g/s)')
68 axs[0].xaxis.set_ticklabels([])
69 plot0 = axs[0].imshow(xpx_dist, vmax=3*density/pxLim,
70                     extent=[xmin,xmax,-pxLim,pxLim], interpolation='none',
71                     aspect='auto', cmap='YlOrBr')
72 fig.colorbar(plot0, ax=axs[0], location='right')
73
74 def plot_xpx():
75     xpx_dist.fill(0)
76     sim.particle_loop(name='electron', handler=xpx_callback.address,
77                     data_double=pipic.addressof(xpx_dist))
78     plot0.set_data(xpx_dist)
79
80 #-----preparing output of kinetic energy-----

```

```

81 mc2 = electron_mass * light_velocity**2
82 m2c2 = (electron_mass * light_velocity)**2
83 kineticEnergy = np.zeros((1, ), dtype=np.double)
84
85 @cfunc(types.particle_loop_callback)
86 def kineticEn_cb(r, p, w, id, data_double, data_int):
87     data_double[0] += w[0]*mc2*(np.sqrt(1+(p[0]**2+p[1]**2+p[2]**2)/m2c2)-1)
88
89 def getKineticEn(sim):
90     kineticEnergy[0] = 0
91     sim.particle_loop(name='electron', handler=kineticEn_cb.address,
92                       data_double=pipic.addressof(kineticEnergy))
93     return kineticEnergy[0]
94
95 #-----preparing output of field energy-----
96 factor = ((sim.xmax-sim.xmin)/sim.nx)*((sim.ymax-sim.ymin)/sim.ny)* \
97          ((sim.zmax-sim.zmin)/sim.nz)/(8*np.pi)
98 field_energy = np.zeros((1, ), dtype=np.double)
99
100 @cfunc(types.field_loop_callback)
101 def fieldEn_cb(ind, r, E, B, data_double, data_int):
102     data_double[0] += factor*(E[0]**2+E[1]**2+E[2]**2+B[0]**2+B[1]**2+B[2]**2)
103
104 def getFieldEnergy(sim):
105     field_energy[0] = 0
106     sim.field_loop(handler=fieldEn_cb.address,
107                   data_double=pipic.addressof(field_energy))
108     return field_energy[0]
109
110
111 #=====SIMULATION=====
112 n_plasma_periods = 100 # number of plasma oscillations to be simulated
113 experiment_duration = n_plasma_periods*plasma_period
114 n_iterations = n_plasma_periods*iterations_period
115 timeP = np.zeros((0, ), dtype=np.double)
116 Efield = np.zeros((0, ), dtype=np.double)
117 Etot = np.zeros((0, ), dtype=np.double)
118 E0 = 0
119 for i in range(n_plasma_periods*iterations_period):
120     print(i, "/", n_plasma_periods*iterations_period)
121     Ek = getKineticEn(sim)
122     Ef = getFieldEnergy(sim)
123     if i == 0:
124         E0 = Ek+Ef
125     timeP = np.append(timeP, i*time_step/plasma_period)
126     Efield = np.append(Efield, Ef/E0)
127     Etot = np.append(Etot, (Ek+Ef)/E0)
128     print('relative energy deviation = ', ((Ek+Ef)-E0)/E0)
129     if Ef > 2*E0:
130         break
131     sim.advance(time_step=time_step)
132
133
134 #=====SAVING SIMULATION RESULT TO FILE=====
135 name = fig_label + '_' + str(iterations_period)
136 with open(name + '.npy', 'wb') as f:
137     np.save(f, Etot)
138     np.save(f, Efield)
139     np.save(f, timeP)

```

Appendix C. Python code used for Sec. 8

```

1  import pipic
2  from pipic import consts, types
3  import matplotlib as mpl
4  import matplotlib.pyplot as plt
5  from mpl_toolkits.axes_grid1 import make_axes_locatable
6  import numpy as np
7  from numba import cfunc, carray, types as nbt
8  import os
9  from pipic.consts import electron_mass, electron_charge, light_velocity
10
11
12  #=====SIMULATION INITIALIZATION=====
13  wavelength = 1e-4
14  nx, xmin, xmax = 128, -8*wavelength, 8*wavelength
15  ny, ymin, ymax = 64, -4*wavelength, 4*wavelength
16  dx, dy = (xmax - xmin)/nx, (ymax - ymin)/ny
17  time_step = 0.5 * dx / light_velocity
18  figStride = int(nx/16)
19
20  @cfunc(nbt.double(nbt.double, nbt.double, nbt.double)) #auxiliary function
21  def cos2shape(x, plateauSize, transitionSize):
22      return (abs(x) < plateauSize/2 + transitionSize) * \
23             (1 - (abs(x) > plateauSize/2) *
24              np.sin(0.5*np.pi*(abs(x) - plateauSize/2)/transitionSize)**2)
25
26  #-----setting solver and simulation region-----
27  sim=pipic.init(solver='ec2', nx=nx, ny=ny, xmin=xmin, xmax=xmax, ymin=ymin, ymax=ymax)
28
29  #-----setting field of the pulse-----
30  amplitude_a0 = 100
31  omega = 2 * np.pi * light_velocity / wavelength
32  a0 = electron_mass * light_velocity * omega / (-electron_charge)
33  fieldAmplitude = amplitude_a0*a0
34  arrivalDelay = 6*2*np.pi/omega
35  incidenceAngle = np.pi/3
36  # setup for a basic field generator/absorber
37  boundarySize = wavelength
38  supRate = 1 - np.exp(np.log(0.01) / (wavelength / (light_velocity * time_step)))
39
40  @cfunc(nbt.double(nbt.double, nbt.double)) # longitudinal shape
41  def pulse(eta, phi):
42      return (abs(eta-2*np.pi)< 2*np.pi)*np.sin(eta/4)**3* \
43             (np.cos(eta/4)*np.sin(eta+phi)+np.sin(eta/4)*np.cos(eta+phi))
44
45  @cfunc(types.field_loop_callback)
46  def field_callback(ind, r, E, B, data_double, data_int):
47      if data_int[0] == 0 or ymax - r[1] < boundarySize:
48          # computing longitudinal and transverse coordinates:
49          lCoord = np.sin(incidenceAngle) * r[0] - np.cos(incidenceAngle) * r[1] + \
50                  + (arrivalDelay - data_int[0]*time_step) * light_velocity
51          tCoord = np.cos(incidenceAngle)*r[0] + np.sin(incidenceAngle)*r[1]
52          tShape = cos2shape(tCoord, 3*wavelength, 2*wavelength)
53          # computing both field components of the CP pulse
54          f_1=fieldAmplitude*tShape*pulse(2*np.pi*lCoord/wavelength+4*np.pi,np.pi)
55          f_2=fieldAmplitude*tShape*pulse(2*np.pi*lCoord/wavelength+4*np.pi,np.pi/2)
56          Ex = np.cos(incidenceAngle)*f_1
57          Ey = np.sin(incidenceAngle)*f_1
58          Bz = f_1
59          Bx = -np.cos(incidenceAngle)*f_2
60          By = -np.sin(incidenceAngle)*f_2
61          Ez = f_2
62          if data_int[0] == 0: # setting initial field

```

```

63     E[0], E[1], E[2], B[0], B[1], B[2] = Ex, Ey, Ez, Bx, By, Bz
64     else: # soft reduction of the field difference (pulse generation)
65         rate = supRate*cos2shape(ymax - r[1], 0, boundarySize)
66         E[0] = (1 - rate)*E[0] + rate*Ex
67         E[1] = (1 - rate)*E[1] + rate*Ey
68         E[2] = (1 - rate)*E[2] + rate*Ez
69         B[0] = (1 - rate)*B[0] + rate*Bx
70         B[1] = (1 - rate)*B[1] + rate*By
71         B[2] = (1 - rate)*B[2] + rate*Bz
72     if r[1] - ymin < boundarySize: # basic absorber
73         rate = supRate*cos2shape(r[1] - ymin, 0, boundarySize)
74         E[0] = (1 - rate)*E[0]
75         E[1] = (1 - rate)*E[1]
76         E[2] = (1 - rate)*E[2]
77         B[0] = (1 - rate)*B[0]
78         B[1] = (1 - rate)*B[1]
79         B[2] = (1 - rate)*B[2]
80
81     #-----setting plasma-----
82     N_cr = electron_mass * omega ** 2 / (4 * np.pi * electron_charge ** 2)
83     density = 100*N_cr
84     debye_length = 0.08165*wavelength/64.0
85     temperature = 4 * np.pi * density * (electron_charge ** 2) * debye_length ** 2
86     particlesPerCell = 100
87
88     @cfunc(types.add_particles_callback)
89     def density_callback(r, data_double, data_int):# callback function
90         return density*cos2shape(r[1] + wavelength, wavelength, wavelength)
91     sim.add_particles(name='electron', number=int(nx*ny*0.25*particlesPerCell),
92                     charge=-electron_charge, mass=electron_mass,
93                     temperature=temperature, density=density_callback.address)
94
95
96     #=====OUTPUT=====
97     fig, axs = plt.subplots(2, constrained_layout=True)
98     ax = axs[0]
99
100    #-----field output-----
101    tmp = np.zeros((1, 1), dtype=np.double) # null plot to show the color bar
102    G_cmap=mpl.colors.LinearSegmentedColormap.from_list('N', [(1,1,1), (0,0.5,0)],N=32)
103    cbar = ax.imshow(tmp, vmin=0, vmax=2,
104                    extent=[xmin,xmax,ymin,ymax], interpolation='none',
105                    aspect='equal', origin='lower', cmap=G_cmap)
106    oBz = np.zeros((ny, nx), dtype=np.double)
107    maxField = 2*fieldAmplitude
108    plot0 = ax.imshow(oBz, vmin=-maxField, vmax=maxField,
109                    extent=[xmin,xmax,ymin,ymax], interpolation='none',
110                    aspect='equal', cmap='RdBu', origin='lower')
111    ax.set(xlabel='$x$ (cm)', ylabel='$y$ (cm)')
112    ax.ticklabel_format(axis='both', scilimits=(0,0), useMathText=True)
113    divider = make_axes_locatable(ax)
114    cax = divider.append_axes("right", size="2%", pad=0.05)
115    fig.add_axes(cax)
116    bar0 = fig.colorbar(plot0, cax=cax, orientation="vertical")
117    bar0.set_label(label='$B_z$ (CGS)')
118
119    @cfunc(types.field_loop_callback)
120    def fieldBz_cb(ind, r, E, B, data_double, data_int):
121        Bz = carray(data_double, oBz.shape, dtype=np.double)
122        Bz[ind[1], ind[0]] = B[2]
123
124    def plot_field():
125        sim.field_loop(handler=fieldBz_cb.address, data_double=pipic.addressof(oBz))

```



```

126     plot0.set_data(oBz)
127
128     #-----density output-----
129     oN = np.zeros((ny, nx), dtype=np.double)
130
131     @cfunc(types.particle_loop_callback)
132     def density_cb(r, p, w, id, data_double, data_int):
133         ix = int(oN.shape[1]*(r[0] - xmin)/(xmax - xmin))
134         iy = int(oN.shape[0]*(r[1] - ymin)/(ymax - ymin))
135         data = carray(data_double, oN.shape, dtype=np.double)
136         if 0 <= iy < oN.shape[0] and 0 <= ix < oN.shape[1]:
137             data[iy, ix] += w[0]/(dx*dy*2*density)
138
139     N_cmap=plt.cm.LinearSegmentedColormap.from_list('N', [(0,0.5,0), (0,0.5,0)], N=9)
140     plot1 = ax.imshow(oN, vmin=0, vmax=1, alpha=oN,
141                     extent=[xmin,xmax,ymin,ymax], interpolation='none',
142                     aspect='equal', origin='lower', cmap=N_cmap)
143     ax.set_xlabel('$x$ (cm)', ylabel='$y$ (cm)')
144     ax.ticklabel_format(axis='both', scilimits=(0,0), useMathText=True)
145     cax1 = divider.append_axes("right", size="2%", pad=0.85)
146     fig.add_axes(cax1)
147     bar1 = fig.colorbar(cbar, cax=cax1, orientation="vertical")
148     bar1.set_label(label='$N/N_0$')
149
150     def plot_density():
151         oN.fill(0)
152         sim.particle_loop(name='electron', handler=density_cb.address,
153                         data_double=pipic.addressof(oN))
154         plot1.set_data(oN)
155
156     #-----ldfield output-----
157     oEp = np.zeros((2*nx,), dtype=np.double) #field P-component of the pulse
158     EpSize = 8*wavelength # size of the output
159
160     @cfunc(types.it2r_callback)
161     def E_it2r(it, r, data_double, data_int):
162         lCoord = (it[0] + 0.5)*EpSize/oEp.shape[0]
163         r[0] = lCoord*np.sin(incidenceAngle)
164         r[1] = lCoord*np.cos(incidenceAngle)
165         r[2] = 0
166
167     @cfunc(types.field2data_callback)
168     def get_Ep(it, r, E, B, data_double, data_int):
169         data_double[it[0]] = -E[0]*np.cos(incidenceAngle) + E[1]*np.sin(incidenceAngle)
170
171     @cfunc(types.field2data_callback)
172     def get_Es(it, r, E, B, data_double, data_int):
173         data_double[it[0]] = E[2]
174
175     axs[1].set_xlim([0, EpSize])
176     axs[1].set_ylim([-maxField, maxField])
177     axs[1].set_xlabel('$x$ (cm)', ylabel='$E_p$ (cgs units)')
178     x_axis = np.linspace(0, EpSize, oEp.shape[0])
179     plot_Ep, = axs[1].plot(x_axis, oEp)
180
181     with open('res.npy', 'rb') as f: # reading data of RES computation
182         res_X_0 = np.load(f)
183         res_Ep = -np.load(f)
184         res_Es = -np.load(f)
185
186     res_X = res_X_0 + -arrivalDelay * light_velocity
187     plot_res_Ep, = axs[1].plot(res_X, res_Ep)
188

```

```

189 def plot_E(i):
190     sim.custom_field_loop(number_of_iterations=oEp.shape[0], it2r=E_it2r.address,
191                           field2data=get_Ep.address,
192                           data_double=pipic.addressof(oEp))
193     plot_Ep.set_ydata(oEp)
194     res_X = res_X_0 + ((-arrivalDelay + i*time_step) * light_velocity +
195                       wavelength * np.cos(incidenceAngle))
196     # this is due to surface being at 0.5 \mu m
197     plot_res_Ep.set_xdata(res_X)
198     if i == 21*figStride:
199         with open('im21_Ep_' + str(nx) + '.npy', 'wb') as f:
200             np.save(f, x_axis)
201             np.save(f, oEp)
202
203
204     #=====SIMULATION=====
205     outputFolder = 'laser_solid_interaction_output'
206     if not os.path.exists(outputFolder):
207         os.makedirs(outputFolder)
208     data_int = np.zeros((1, ), dtype=np.intc) # data for passing the iteration number
209     for i in range(figStride*21 + 1):
210         print(i, '/', figStride*21 + 1)
211         data_int[0] = i
212         sim.field_loop(handler=field_callback.address,
213                       data_int=pipic.addressof(data_int), use_omp=True)
214         if i%figStride == 0:
215             plot_field()
216             plot_density()
217             plot_E(i)
218             fig.savefig(outputFolder+'im'+str(int(i/figStride))+'.png', dpi=300)
219         sim.advance(time_step=time_step)

```

References

- [1] C.K. Birdsall, A.B. Langdon, *Plasma Physics via Computer Simulation*, McGraw Hill Higher Education, Maidenhead, England, 1984.
- [2] R.A. Fonseca, L.O. Silva, F.S. Tsung, V.K. Decyk, W. Lu, C. Ren, W.B. Mori, S. Deng, S. Lee, T. Katsouleas, J.C. Adam, OSIRIS: a three-dimensional, fully relativistic particle in cell code for modeling plasma based accelerators, in: *Lecture Notes in Computer Science*, Springer, Berlin Heidelberg, 2002, pp. 342–351.
- [3] S. Bastrakov, R. Donchenko, A. Gonoskov, E. Efimenko, A. Malyshev, I. Meyerov, I. Surmin, Particle-in-cell plasma simulation on heterogeneous cluster systems, *J. Comput. Sci.* 3 (6) (2012) 474–479, <https://doi.org/10.1016/j.jocs.2012.08.012>.
- [4] M. Bussmann, H. Burau, T.E. Cowan, A. Debus, A. Huebl, G. Juckeland, T. Kluge, W.E. Nagel, R. Pausch, F. Schmitt, U. Schramm, J. Schuchart, R. Widera, Radiative signatures of the relativistic Kelvin-Helmholtz instability, in: *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, ACM, 2013.
- [5] J. Derouillat, A. Beck, F. Pérez, T. Vinci, M. Chiaramello, A. Grassi, M. Flé, G. Bouchard, I. Plotnikov, N. Aunai, J. Dargent, C. Riconda, M. Grech, Smilei: a collaborative, open-source, multi-purpose particle-in-cell code for plasma simulation, *Comput. Phys. Commun.* 222 (2018) 351–373, <https://doi.org/10.1016/j.cpc.2017.09.024>.
- [6] A. Myers, A. Almgren, L. Amorim, J. Bell, L. Fedeli, L. Ge, K. Gott, D. Grote, M. Hogan, A. Huebl, R. Jambunathan, R. Lehe, C. Ng, M. Rowan, O. Shapoval, M. Thévenet, J.-L. Vay, H. Vincenti, E. Yang, N. Zaïm, W. Zhang, Y. Zhao, E. Zoni, Porting WarpX to GPU-accelerated platforms, *Parallel Comput.* 108 (2021) 102833, <https://doi.org/10.1016/j.parco.2021.102833>.
- [7] R. Bird, N. Tan, S.V. Luedtke, S.L. Harrell, M. Taufer, B. Albright, VPIC 2.0: next generation particle-in-cell simulations, *IEEE Trans. Parallel Distrib. Syst.* 33 (4) (2022) 952–963, <https://doi.org/10.1109/tpds.2021.3084795>.
- [8] L. Fedeli, A. Huebl, F. Boillod-Cerneux, T. Clark, K. Gott, C. Hillairet, S. Jaure, A. Leblanc, R. Lehe, A. Myers, C. Piechurski, M. Sato, N. Zaim, W. Zhang, J. Vay, H. Vincenti, Pushing the frontier in the design of laser-based electron accelerators with groundbreaking mesh-refined particle-in-cell simulations on exascale-class supercomputers, in: 2022 SC22: International Conference for High Performance Computing, Networking, Storage and Analysis (SC) (SC), IEEE Computer Society, Los Alamitos, CA, USA, 2022, pp. 25–36, <https://www.computer.org/csdl/proceedings-article/sc/2022/544400a025/110bSKaoECc>.
- [9] R. Duclous, J.G. Kirk, A.R. Bell, Monte Carlo calculations of pair production in high-intensity laser-plasma interactions, *Plasma Phys. Control. Fusion* 53 (1) (2011) 015009, <https://doi.org/10.1088/0741-3335/53/1/015009>, <http://stacks.iop.org/0741-3335/53/i=1/a=015009?key=crossref.a8a8821fd000c941572db8818a245120>.
- [10] E.N. Nerush, I.Y. Kostyukov, A.M. Fedotov, N.B. Narozhny, N.V. Elkina, H. Ruhl, Laser field absorption in self-generated electron-positron pair plasma, *Phys. Rev. Lett.* 106 (3) (3, 2011), <https://doi.org/10.1103/PhysRevLett.106.035001>.
- [11] N.V. Elkina, A.M. Fedotov, I.Y. Kostyukov, M.V. Legkov, N.B. Narozhny, E.N. Nerush, H. Ruhl, QED cascades induced by circularly polarized laser fields, *Phys. Rev. Spec. Top., Accel. Beams* 14 (5) (2011) 054401, <https://doi.org/10.1103/PhysRevSTAB.14.054401>, <https://link.aps.org/doi/10.1103/PhysRevSTAB.14.054401>.
- [12] I.V. Sokolov, N.M. Naumova, J.A. Nees, Numerical modeling of radiation-dominated and quantum-electrodynamically strong regimes of laser-plasma interaction, *Phys. Plasmas* 18 (9) (2011) 093109, <https://doi.org/10.1063/1.3638138>, <http://aip.scitation.org/doi/10.1063/1.3638138>.
- [13] C.P. Ridgers, J.G. Kirk, R. Duclous, T.G. Blackburn, C.S. Brady, K. Bennett, T.D. Arber, A.R. Bell, Modelling gamma-ray photon emission and pair production in high-intensity laser-matter interactions, *J. Comput. Phys.* 260 (2014) 273–285, <https://doi.org/10.1016/j.jcp.2013.12.007>.
- [14] A. Gonoskov, S. Bastrakov, E. Efimenko, A. Ilderton, M. Marklund, I. Meyerov, A. Muraviev, A. Sergeev, I. Surmin, E. Wallin, Extended particle-in-cell schemes for physics in ultrastrong laser fields: review and developments, *Phys. Rev. E* 92 (2) (2015) 023305.
- [15] T.D. Arber, K. Bennett, C.S. Brady, A. Lawrence-Douglas, M.G. Ramsay, N.J. Sircombe, P. Gillies, R.G. Evans, H. Schmitz, A.R. Bell, C.P. Ridgers, Contemporary particle-in-cell approach to laser-plasma modelling, *Plasma Phys. Control. Fusion* 57 (11) (2015) 113001, <https://doi.org/10.1088/0741-3335/57/11/113001>.

- [16] M. Lobet, E. D'Humières, M. Grech, C. Ruyer, X. Davoine, L. Gremillet, Modeling of radiative and quantum electrodynamics effects in PIC simulations of ultra-relativistic laser-plasma interaction, *J. Phys. Conf. Ser.* 688 (2016) 12058, <https://doi.org/10.1088/1742-6596/688/1/012058>, <https://iopscience.iop.org/article/10.1088/1742-6596/688/1/012058/meta>.
- [17] S. Rjasanow, W. Wagner, A stochastic weighted particle method for the Boltzmann equation, *J. Comput. Phys.* 124 (2) (1996) 243–253, <https://doi.org/10.1006/jcph.1996.0057>.
- [18] S. Rjasanow, T. Schreiber, W. Wagner, Reduction of the number of particles in the stochastic weighted particle method for the Boltzmann equation, *J. Comput. Phys.* 145 (1) (1998) 382–405, <https://doi.org/10.1006/jcph.1998.6018>.
- [19] G. Lapenta, J.U. Brackbill, Dynamic and selective control of the number of particles in kinetic plasma simulations, *J. Comput. Phys.* 115 (1) (1994) 213–227, <https://doi.org/10.1006/jcph.1994.1188>.
- [20] G. Lapenta, J.U. Brackbill, Control of the number of particles in fluid and MHD particle in cell methods, *Comput. Phys. Commun.* 87 (1–2) (1995) 139–154, [https://doi.org/10.1016/0010-4655\(94\)00180-A](https://doi.org/10.1016/0010-4655(94)00180-A).
- [21] F. Assous, T. Pougeard Dulimbert, J. Segré, A new method for coalescing particles in PIC codes, *J. Comput. Phys.* 187 (2) (2003) 550–571, [https://doi.org/10.1016/S0021-9991\(03\)00124-4](https://doi.org/10.1016/S0021-9991(03)00124-4).
- [22] D.R. Welch, T.C. Genoni, R.E. Clark, D.V. Rose, Adaptive particle management in a particle-in-cell code, *J. Comput. Phys.* 227 (1) (2007) 143–155, <https://doi.org/10.1016/j.jcp.2007.07.015>.
- [23] A.N. Timokhin, Time-dependent pair cascades in magnetospheres of neutron stars - I. Dynamics of the polar cap cascade with no particle supply from the neutron star surface, *Mon. Not. R. Astron. Soc.* 408 (4) (2010) 2092–2114, <https://doi.org/10.1111/j.1365-2966.2010.17286.x>, <https://academic.oup.com/mnras/article/408/4/2092/1418274>.
- [24] M. Vranic, T. Grismayer, J.L. Martins, R.A. Fonseca, L.O. Silva, Particle merging algorithm for PIC codes, *Comput. Phys. Commun.* 191 (1) (2015) 65–73, <https://doi.org/10.1016/j.cpc.2015.01.020>.
- [25] M. Pfeiffer, A. Mirza, C.D. Munz, S. Fasoulas, Two statistical particle split and merge methods for particle-in-cell codes, *Comput. Phys. Commun.* 191 (1) (2015) 9–24, <https://doi.org/10.1016/j.cpc.2015.01.010>.
- [26] R.S. Martin, J.L. Cambier, Octree particle management for DSMC and PIC simulations, *J. Comput. Phys.* 327 (2016) 943–966, <https://doi.org/10.1016/j.jcp.2016.01.020>.
- [27] P.T. Luu, T. Tückmantel, A. Pukhov, Voronoi particle merging algorithm for PIC codes, *Comput. Phys. Commun.* 202 (2016) 165–174, <https://doi.org/10.1016/j.cpc.2016.01.009>.
- [28] D. Faghihi, V. Carey, C. Michoski, R. Hager, S. Janhunen, C.S. Chang, R.D. Moser, Moment preserving constrained resampling with applications to particle-in-cell methods, *J. Comput. Phys.* 409 (2020) 109317, <https://doi.org/10.1016/j.jcp.2020.109317>.
- [29] A. Muraviev, A. Bashinov, E. Efimenko, V. Volokitin, I. Meyerov, A. Gonoskov, Strategies for particle resampling in PIC simulations, *Comput. Phys. Commun.* 262 (2021) 107826, <https://doi.org/10.1016/j.cpc.2021.107826>.
- [30] A. Gonoskov, Agnostic conservative down-sampling for optimizing statistical representations and PIC simulations, *Comput. Phys. Commun.* 271 (2022) 108200, <https://doi.org/10.1016/j.cpc.2021.108200>.
- [31] I. Haber, R. Lee, H. Klein, J. Boris, *Advances in electromagnetic simulation techniques*, in: *Proc. Sixth Conf. Num. Sim. Plasmas*, Berkeley, CA, 1973, pp. 46–48.
- [32] O. Buneman, C. Barnes, J. Green, D. Nielsen, Principles and capabilities of 3-d, e-m particle simulations, *J. Comput. Phys.* 38 (1) (1980) 1–44, [https://doi.org/10.1016/0021-9991\(80\)90010-8](https://doi.org/10.1016/0021-9991(80)90010-8).
- [33] J.-L. Vay, I. Haber, B.B. Godfrey, A domain decomposition method for pseudo-spectral electromagnetic simulations of plasmas, *J. Comput. Phys.* 243 (2013) 260–268, <https://doi.org/10.1016/j.jcp.2013.03.010>.
- [34] A. Gonoskov, Ultra-intense laser-plasma interaction for applied and fundamental physics, Ph.D. thesis, Umeå University, 2013, <http://www.diva-portal.org/smash/record.jsf?pid=diva2%3A681092&dsid=-7208>.
- [35] K. Yee, Numerical solution of initial boundary value problems involving Maxwell's equations in isotropic media, *IEEE Trans. Antennas Propag.* 14 (3) (1966) 302–307, <https://doi.org/10.1109/tap.1966.1138693>.
- [36] A. Pukhov, Three-dimensional electromagnetic relativistic particle-in-cell code VLPL (virtual laser plasma lab), *J. Plasma Phys.* 61 (3) (1999) 425–433, <https://doi.org/10.1017/s0022377899007515>.
- [37] B.M. Cowan, D.L. Bruhwiler, J.R. Cary, E. Cormier-Michel, C.G.R. Geddes, Generalized algorithm for control of numerical dispersion in explicit time-domain electromagnetic simulations, *Phys. Rev. Spec. Top., Accel. Beams* 16 (4) (Apr. 2013), <https://doi.org/10.1103/physrevstab.16.041303>.
- [38] R. Lehe, A. Lifschitz, C. Thaur, V. Malka, X. Davoine, Numerical growth of emittance in simulations of laser-wakefield acceleration, *Phys. Rev. Spec. Top., Accel. Beams* 16 (2) (Feb. 2013), <https://doi.org/10.1103/physrevstab.16.021301>.
- [39] A. Blinne, D. Schinkel, N. Kuschel, N. Elkina, S.G. Rykovanov, M. Zepf, A systematic approach to numerical dispersion in Maxwell solvers, *Comput. Phys. Commun.* 224 (2018) 273–281, <https://doi.org/10.1016/j.cpc.2017.10.010>.
- [40] A. Pukhov, X-dispersionless Maxwell solver for plasma-based particle acceleration, *J. Comput. Phys.* 418 (2020) 109622, <https://doi.org/10.1016/j.jcp.2020.109622>.
- [41] H. Lewis, Energy-conserving numerical approximations for Vlasov plasmas, *J. Comput. Phys.* 6 (1) (1970) 136–141, [https://doi.org/10.1016/0021-9991\(70\)90012-4](https://doi.org/10.1016/0021-9991(70)90012-4).
- [42] A.B. Langdon, “Energy-conserving” plasma simulation algorithms, *J. Comput. Phys.* 12 (2) (1973) 247–268, [https://doi.org/10.1016/s0021-9991\(73\)80014-2](https://doi.org/10.1016/s0021-9991(73)80014-2).
- [43] H. Kim, L. Chacon, G. Lapenta, Fully implicit particle-in-cell algorithm, in: *Annual DPP Meeting*, Date: 2005/10/24 - 2005/10/28, Location: Denver, Colorado, 2005.
- [44] S. Markidis, Development of implicit kinetic simulation methods, and their application to ion beam propagation in current and future neutralized drift compression experiments, Ph.D. thesis, University of Illinois at Urbana-Champaign, 2010.
- [45] S. Markidis, G. Lapenta, The energy conserving particle-in-cell method, *J. Comput. Phys.* 230 (18) (2011) 7037–7052, <https://doi.org/10.1016/j.jcp.2011.05.033>.
- [46] G. Chen, L. Chacón, D. Barnes, An energy- and charge-conserving, implicit, electrostatic particle-in-cell algorithm, *J. Comput. Phys.* 230 (18) (2011) 7018–7036, <https://doi.org/10.1016/j.jcp.2011.05.031>.
- [47] W.T. Taitano, D.A. Knoll, L. Chacón, G. Chen, Development of a consistent and stable fully implicit moment method for Vlasov–Ampère particle in cell (PIC) system, *SIAM J. Sci. Comput.* 35 (5) (2013) S126–S149, <https://doi.org/10.1137/120881385>.
- [48] G. Chen, L. Chacón, L. Yin, B. Albright, D. Stark, R. Bird, A semi-implicit, energy- and charge-conserving particle-in-cell algorithm for the relativistic Vlasov–Maxwell equations, *J. Comput. Phys.* 407 (2020) 109228, <https://doi.org/10.1016/j.jcp.2020.109228>.
- [49] Y. Li, Energy conserving particle-in-cell methods for relativistic Vlasov–Maxwell equations of laser-plasma interaction, *arXiv:2203.06513*, 2022.
- [50] G. Lapenta, Exactly energy conserving semi-implicit particle in cell formulation, *J. Comput. Phys.* 334 (2017) 349–366, <https://doi.org/10.1016/j.jcp.2017.01.002>.
- [51] J. Brackbill, D. Forslund, An implicit method for electromagnetic plasma simulation in two dimensions, *J. Comput. Phys.* 46 (2) (1982) 271–308, [https://doi.org/10.1016/0021-9991\(82\)90016-x](https://doi.org/10.1016/0021-9991(82)90016-x).
- [52] H. Vu, J. Brackbill, CELEST1d: an implicit, fully kinetic model for low-frequency, electromagnetic plasma simulation, *Comput. Phys. Commun.* 69 (2–3) (1992) 253–276, [https://doi.org/10.1016/0010-4655\(92\)90165-u](https://doi.org/10.1016/0010-4655(92)90165-u).
- [53] A. Langdon, B.I. Cohen, A. Friedman, Direct implicit large time-step particle simulation of plasmas, *J. Comput. Phys.* 51 (1) (1983) 107–138, [https://doi.org/10.1016/0021-9991\(83\)90083-9](https://doi.org/10.1016/0021-9991(83)90083-9).

- [54] D.W. Hewett, A.B. Langdon, Electromagnetic direct implicit plasma simulation, *J. Comput. Phys.* 72 (1) (1987) 121–155, [https://doi.org/10.1016/0021-9991\(87\)90075-1](https://doi.org/10.1016/0021-9991(87)90075-1).
- [55] G. Strang, On the construction and comparison of difference schemes, *SIAM J. Numer. Anal.* 5 (3) (1968) 506–517, <https://doi.org/10.1137/0705041>.
- [56] R.I. McLachlan, G.R.W. Quispel, Splitting methods, *Acta Numer.* 11 (2002) 341–434, <https://doi.org/10.1017/s0962492902000053>.
- [57] D. Barnes, L. Chacón, Finite spatial-grid effects in energy-conserving particle-in-cell algorithms, *Comput. Phys. Commun.* 258 (2021) 107560, <https://doi.org/10.1016/j.cpc.2020.107560>.
- [58] K. Tangtartharakul, G. Chen, A. Arefiev, Particle integrator for particle-in-cell simulations of ultra-high intensity laser-plasma interactions, *J. Comput. Phys.* 434 (2021) 110233, <https://doi.org/10.1016/j.jcp.2021.110233>.
- [59] J.-L. Vay, Simulation of beams or plasmas crossing at relativistic velocity, *Phys. Plasmas* 15 (5) (2008) 056701, <https://doi.org/10.1063/1.2837054>.
- [60] A.V. Arefiev, G.E. Cochran, D.W. Schumacher, A.P.L. Robinson, G. Chen, Temporal resolution criterion for correctly simulating relativistic electron motion in a high-intensity laser field, *Phys. Plasmas* 22 (1) (2015) 013103, <https://doi.org/10.1063/1.4905523>.
- [61] J.P. Boris, et al., Relativistic plasma simulation-optimization of a hybrid code, in: *Proc. Fourth Conf. Num. Sim. Plasmas*, 1970, pp. 3–67.
- [62] T. Esirkepov, Exact charge conservation scheme for particle-in-cell simulation with an arbitrary form-factor, *Comput. Phys. Commun.* 135 (2) (2001) 144–153, [https://doi.org/10.1016/s0010-4655\(00\)00228-9](https://doi.org/10.1016/s0010-4655(00)00228-9).
- [63] J.V. Jose, E.J. Saletan, *Classical Dynamics*, Cambridge University Press, Cambridge, England, 1998.
- [64] J.P. Boris, R.A. Shanny, in: *Proceedings [of the] 4th Conference on Numerical Simulation of Plasmas*, Naval Research Laboratory, 1972.
- [65] Q.H. Liu, The PSTD algorithm: a time-domain method requiring only two cells per wavelength, *Microw. Opt. Technol. Lett.* 15 (3) (1997) 158–165, [https://doi.org/10.1002/\(sici\)1098-2760\(19970620\)15:3<158::aid-mop11>3.0.co;2-3](https://doi.org/10.1002/(sici)1098-2760(19970620)15:3<158::aid-mop11>3.0.co;2-3).
- [66] R. Lehe, M. Kirchen, I.A. Andriyash, B.B. Godfrey, J.-L. Vay, A spectral, quasi-cylindrical and dispersion-free particle-in-cell algorithm, *Comput. Phys. Commun.* 203 (2016) 66–82, <https://doi.org/10.1016/j.cpc.2016.02.007>.
- [67] π -PIC, open-source PIC code, Copyright 2023 Arkady Gonoskov, <https://github.com/hi-chi/pipic>.
- [68] hi- γ project, <https://github.com/hi-chi>.
- [69] M. Frigo, S. Johnson, The design and implementation of FFTW3, *Proc. IEEE* 93 (2) (2005) 216–231, <https://doi.org/10.1109/jproc.2004.840301>.
- [70] E. Panova, V. Volokitin, E. Efimenko, J. Ferri, T. Blackburn, M. Marklund, A. Muschet, A.D.A. Gonzalez, P. Fischer, L. Veisz, I. Meyerov, A. Gonoskov, Optimized computation of tight focusing of short pulses using mapping to periodic space, *Appl. Sci.* 11 (3) (2021) 956, <https://doi.org/10.3390/app11030956>.
- [71] C. Cheng, G. Knorr, The integration of the Vlasov equation in configuration space, *J. Comput. Phys.* 22 (3) (1976) 330–351, [https://doi.org/10.1016/0021-9991\(76\)90053-x](https://doi.org/10.1016/0021-9991(76)90053-x).
- [72] M. Shalaby, A.E. Broderick, P. Chang, C. Pfrommer, A. Lamberts, E. Puchwein, Importance of resolving the spectral support of beam-plasma instabilities in simulations, *Astrophys. J.* 848 (2) (2017) 81, <https://doi.org/10.3847/1538-4357/aa8b17>.
- [73] A.A. Gonoskov, A.V. Korzhimanov, A.V. Kim, M. Marklund, A.M. Sergeev, Ultrarelativistic nanoplasmonics as a route towards extreme-intensity attosecond pulses, *Phys. Rev. E* 84 (2011) 046403, <https://doi.org/10.1103/PhysRevE.84.046403>, <https://link.aps.org/doi/10.1103/PhysRevE.84.046403>.
- [74] A. Gonoskov, Theory of relativistic radiation reflection from plasmas, *Phys. Plasmas* 25 (1) (2018) 013108, <https://doi.org/10.1063/1.5000785>, <http://aip.scitation.org/doi/10.1063/1.5000785>.
- [75] A. Macchi, M. Borghesi, M. Passoni, Ion acceleration by superintense laser-plasma interaction, *Rev. Mod. Phys.* 85 (2) (2013) 751–793, <https://doi.org/10.1103/revmodphys.85.751>.
- [76] E. Esarey, C.B. Schroeder, W.P. Leemans, Physics of laser-driven plasma-based electron accelerators, *Rev. Mod. Phys.* 81 (3) (2009) 1229–1285, <https://doi.org/10.1103/revmodphys.81.1229>.
- [77] P. Yu, X. Xu, V.K. Decyk, W. An, J. Vieira, F.S. Tsung, R.A. Fonseca, W. Lu, L.O. Silva, W.B. Mori, Modeling of laser wakefield acceleration in Lorentz boosted frame using EM-PIC code with spectral solver, *J. Comput. Phys.* 266 (2014) 124–138, <https://doi.org/10.1016/j.jcp.2014.02.016>.
- [78] F. Fiuza, M. Marti, R.A. Fonseca, L.O. Silva, J. Tonge, J. May, W.B. Mori, Efficient modeling of laser-plasma interactions in high energy density scenarios, *Plasma Phys. Control. Fusion* 53 (7) (2011) 074004, <https://doi.org/10.1088/0741-3335/53/7/074004>.